Kubernetes HA Deployment Runbook

Version 1.0

External etcd + kubeadm HA Cluster

Editorial Review Notes

This runbook was compiled from a longer set of working notes. Before formatting it as a PDF, it was cleaned up and technically reviewed. Summary of what changed:

Cleanup

- Removed conversational scaffolding (“Perfect, this is exactly how we’ll build it,” etc.) that didn’t carry technical content, and three duplicated section-title headers left over from that scaffolding.
- Removed an entire duplicated copy of Stage 3 (Install and Configure containerd) — the source notes contained the same ~600-line stage pasted back to back, word for word.
- Removed an early, superseded outline of the stage numbering (a 12-stage plan that doesn’t match the 0–10 structure the runbook actually ends up using), to avoid confusing readers with a table of contents that doesn’t reflect the document.

Technical corrections

- **Missing stage found:** Stage 2 lists `kubeadm`, `kubelet`, and `kubectl` as node prerequisites, and the plan even calls out a future “Kubernetes Packages” stage, but no install commands for those three packages ever appear in the source notes. A new **Stage 2.9** has been added (clearly marked as an editorial addition) with the standard `pkgs.k8s.io` repository setup, version-pinned install, and `apt-mark hold` — matching the `v1.31.14` version and `containerd` runtime already committed to elsewhere in the document.
- **`kubeadm-config.yaml` fixed for its own declared API version:** the file declares `apiVersion: kubeadm.k8s.io/v1beta4`, but `apiServer.extraArgs` was written using the older `v1beta3` flat-map syntax (`key: value`). Kubernetes 1.31’s `v1beta4` API changed `extraArgs` to a structured list of `name` / `value` pairs; the original map syntax would fail config validation under `v1beta4`. Corrected in Stage 8.2, with a note in place.
- The existing note in Stage 10.9 flagging `kubectl get componentstatuses` as deprecated (in favor of `kubectl get --raw='/readyz?verbose'`) was already accurate and left as-is.

What wasn’t changed: the overall architecture (external etcd, HAProxy/Keepalived VIP, manually issued PKI feeding into kubeadm, Calico networking), the IP plan, and the command sequences were technically sound throughout and are reproduced as designed.

Stage 0 — Cluster Planning

Objective

Before installing Kubernetes, we must design the architecture.

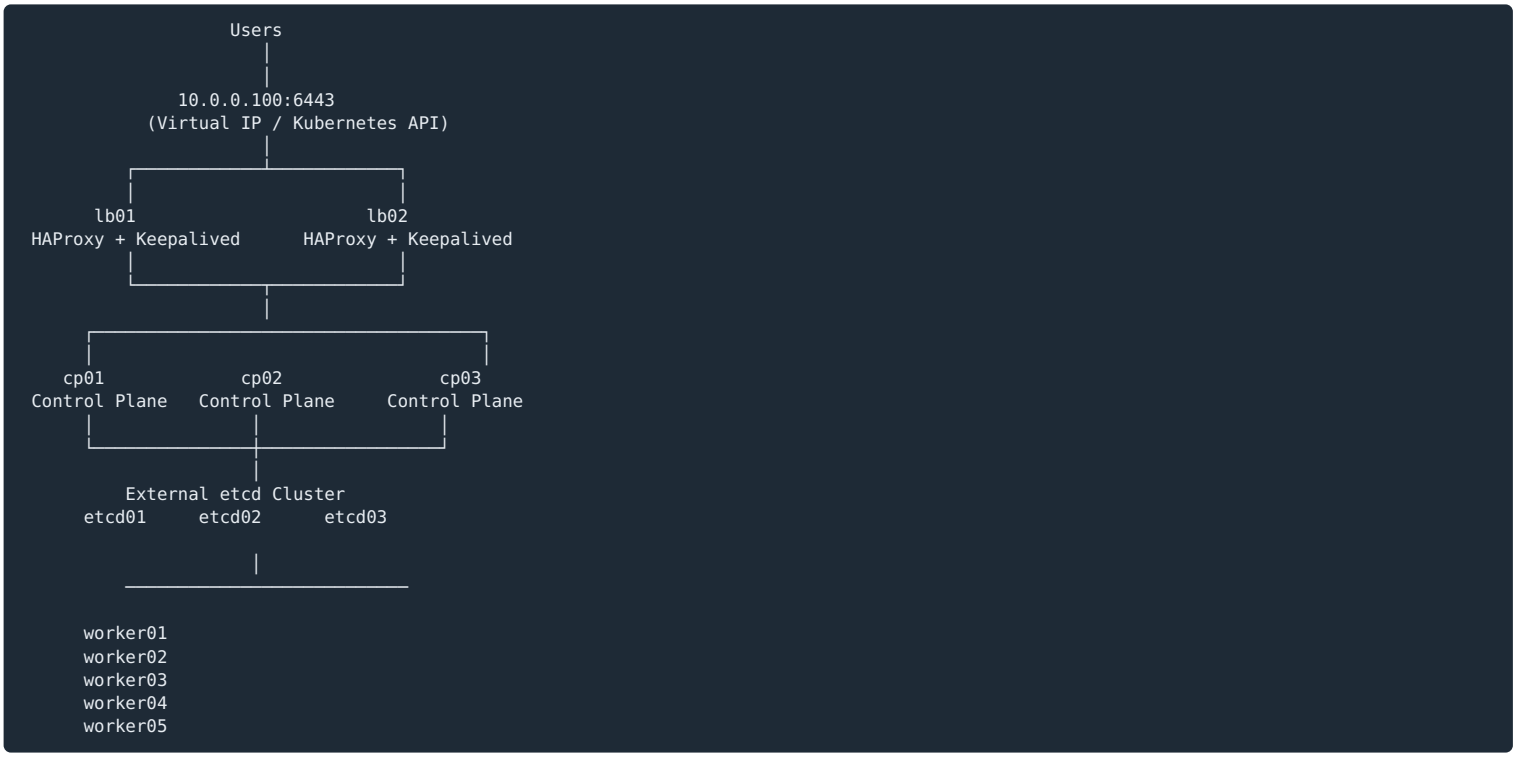
A Kubernetes cluster is **not just several VMs**.

Every component depends on IP addresses, certificates, networking, storage, and high availability.

If these are not planned first, the cluster becomes difficult to scale or troubleshoot.

This stage defines the complete production architecture.

Cluster Architecture



Why use this architecture?

Imagine your Kubernetes API runs only on cp01.

```
Worker ---> cp01
```

If cp01 crashes,

the entire cluster becomes unavailable.

No new Pods can be scheduled.

No kubectl commands work.

No deployments can happen.

Instead, Kubernetes HA works like this.

Workers never contact cp01 directly.

Instead they contact

```
10.0.0.100:6443
```

That is the Virtual IP (VIP).

HAProxy automatically forwards requests to whichever control-plane node is healthy.

If cp01 dies,

HAProxy immediately sends traffic to cp02.

Workers never notice.

Why Keepalived?

HAProxy itself can fail.

Suppose lb01 crashes.

Without Keepalived,

the VIP disappears.

Your entire cluster becomes unreachable.

Keepalived solves this.

It moves

```
10.0.0.100
```

from

```
lb01
```

to

```
lb02
```

within seconds.

Workers continue using

```
10.0.0.100:6443
```

without any changes.

Why External etcd?

etcd stores the Kubernetes database.

It contains:

- Pods
- Deployments
- Secrets
- ConfigMaps
- Nodes
- RBAC
- Certificates

Everything.

If etcd is lost,

the cluster is lost.

Running etcd on separate nodes provides:

- better performance
- easier backups
- isolation
- production-grade architecture

Cluster Nodes

Node	Purpose
lb01	HAProxy + Keepalived
lb02	HAProxy + Keepalived
cp01	Kubernetes Control Plane
cp02	Kubernetes Control Plane
cp03	Kubernetes Control Plane
etcd01	External etcd
etcd02	External etcd
etcd03	External etcd
worker01	Worker
worker02	Worker

worker03	Worker
worker04	Worker
worker05	Worker

IP Address Plan

Node	IP
VIP	10.0.0.100
lb01	10.0.0.10
lb02	10.0.0.20
cp01	10.0.0.11
cp02	10.0.0.12
cp03	10.0.0.13
etcd01	10.0.0.21
etcd02	10.0.0.22
etcd03	10.0.0.23
worker01	10.0.0.31
worker02	10.0.0.32
worker03	10.0.0.33
worker04	10.0.0.34
worker05	10.0.0.35

Kubernetes Version

```
v1.31.14
```

Container Runtime

```
containerd
```

Why?

Because Dockershim was removed from Kubernetes.

containerd is the official runtime recommended by Kubernetes.

Network Plugin

```
Calico
```

Why?

- Production ready
- NetworkPolicy support
- High performance
- Widely used
- CKA friendly

Pod Network

```
192.168.0.0/16
```

Why?

Every Pod receives an IP from this range.

Later Calico divides it into smaller PodCIDRs.

Example:

```
worker01
192.168.5.0/26

worker02
192.168.30.64/26

cp01
192.168.214.128/26
```

This is why you later saw routes like:

```
192.168.30.64/26 via 10.0.0.32
```

Those are PodCIDRs assigned by Calico.

Service Network

```
10.96.0.0/12
```

This network is **not** used by Pods.

It is reserved for Kubernetes Services.

Example:

```
kubernetes.default

10.96.0.1
```

That IP is virtual.

It never belongs to a VM.

kube-proxy redirects traffic to the correct backend Pods.

Cluster DNS

```
10.96.0.10
```

CoreDNS runs behind this IP.

Pods resolve names such as

```
nginx.default.svc.cluster.local
```

using CoreDNS.

Cluster Name

```
pico-prod
```

This name appears in kubeconfig files and identifies the cluster.

Certificates

We intentionally used our own CA instead of kubeadm-generated certificates.

Why?

Because this teaches:

- PKI
- TLS
- Kubernetes certificates
- Mutual authentication
- External etcd TLS

Understanding certificates is essential for troubleshooting and for the CKA exam.

Deployment Strategy

We chose:

```
kubeadm
```

instead of manually creating every static pod.

Why?

Manual deployment teaches internals but is rarely used today.

kubeadm is:

- the Kubernetes standard
- officially maintained
- production ready
- easier to upgrade
- used in most organizations

You still learned the certificate generation process manually before letting kubeadm consume those certificates.

This gives you both conceptual understanding and a production-ready deployment.

Stage 0 Outcome

After completing Stage 0, we have:

- ✅ Designed the HA architecture.
- ✅ Selected Kubernetes version.
- ✅ Planned networking.
- ✅ Planned HA with VIP, HAProxy, and Keepalived.
- ✅ Planned external etcd.
- ✅ Decided on containerd and Calico.
- ✅ Defined certificate strategy.
- ✅ Chosen kubeadm as the deployment tool.

No commands are executed in this stage. It is entirely about planning the cluster before installation.

🧠 CKA Concepts Learned

By the end of Stage 0, you should understand:

- Why Kubernetes uses a control plane and worker architecture.
- Why a Virtual IP (VIP) is used instead of pointing workers to a single control-plane node.
- The roles of HAProxy and Keepalived in providing high availability.
- Why external etcd is recommended for production clusters.
- The difference between Pod CIDR and Service CIDR.
- Why containerd is the preferred runtime.
- Why Calico is chosen as the CNI plugin.
- Why certificates are central to Kubernetes security.
- Why kubeadm is the standard deployment tool today.

Stage 1 — Infrastructure Preparation

Objective

Before installing Kubernetes, we must ensure that the underlying infrastructure is correctly designed.

Kubernetes is **not responsible** for creating VMs, networking, or storage. It assumes the infrastructure already exists.

If the infrastructure is unstable, Kubernetes will also be unstable.

Therefore, the first step is to prepare all machines with consistent networking and system resources.

What We Prepare

During this stage, we prepare:

- Virtual Machines
- CPU and Memory
- Disk
- Network
- DNS/Hostname
- SSH Access
- Time Synchronization
- Firewall Rules
- Load Balancers

No Kubernetes software is installed yet.

Cluster Inventory

Node	Role	CPU	RAM	Disk	IP
lb01	HAProxy + Keepalived	2	2 GB	20 GB	10.0.0.10
lb02	HAProxy + Keepalived	2	2 GB	20 GB	10.0.0.20
cp01	Control Plane	4	8 GB	100 GB	10.0.0.11
cp02	Control Plane	4	8 GB	100 GB	10.0.0.12
cp03	Control Plane	4	8 GB	100 GB	10.0.0.13
etcd01	External etcd	2	4 GB	50 GB	10.0.0.21
etcd02	External etcd	2	4 GB	50 GB	10.0.0.22
etcd03	External etcd	2	4 GB	50 GB	10.0.0.23
worker01	Worker	4	8 GB	100 GB	10.0.0.31
worker02	Worker	4	8 GB	100 GB	10.0.0.32
worker03	Worker	4	8 GB	100 GB	10.0.0.33
worker04	Worker	4	8 GB	100 GB	10.0.0.34
worker05	Worker	4	8 GB	100 GB	10.0.0.35

> **Why these specifications?** Control-plane nodes run the API server, scheduler, and controller manager, so they require more CPU and memory than the load balancers. Worker nodes host application workloads. etcd benefits more from fast storage than from many CPU cores.

Network Topology



Every node must be able to communicate with every other node.

Kubernetes is **not NAT-friendly** inside the cluster. Nodes must have direct Layer-3 connectivity.

Why a /16 Network?

You used:

```
10.0.0.0/16
```

instead of

```
10.0.0.0/24
```

This provides over 65,000 IP addresses and gives plenty of room for cluster growth.

Required Network Connectivity

Every node should be able to reach every other node.

Example:

```
cp01  → cp02
cp01  → cp03
cp01  → etcd01
cp01  → worker01
worker03 → cp02
worker05 → etcd03
```

If any of these paths fail, cluster components may fail to communicate.

DNS and Hostnames

Each node should have a unique hostname.

Examples:

```
cp01
cp02
cp03

worker01
worker02

etcd01
```

Why?

Kubernetes certificates include node names. Duplicate or incorrect hostnames can cause TLS authentication failures.

Commands (Run on Every Node)

Check the hostname

```
hostnamectl
```

Expected output:

```
Static hostname: cp01
```

Change the hostname (if required)

```
hostnamectl set-hostname cp01
```

Replace `cp01` with the appropriate hostname for each machine.

Verify the hostname

```
hostname
```

Configure `/etc/hosts`

Run on **every node**.

Edit:

```
vim /etc/hosts
```

Example:

```
10.0.0.10  lb01
10.0.0.20  lb02

10.0.0.11  cp01
10.0.0.12  cp02
10.0.0.13  cp03

10.0.0.21  etcd01
10.0.0.22  etcd02
10.0.0.23  etcd03

10.0.0.31  worker01
10.0.0.32  worker02
10.0.0.33  worker03
10.0.0.34  worker04
10.0.0.35  worker05
```

> **Why `/etc/hosts`?** During the initial deployment we avoid relying on external DNS. Hostname resolution is immediate and predictable. In production, internal DNS may replace this.

Verify Name Resolution

Run on **every node**.

```
ping cp02
```

```
ping etcd01
```

```
ping worker05
```

Each command should resolve the hostname to the correct IP address.

Verify Network Connectivity

From **cp01**:

```
ping -c 4 cp02
```

```
ping -c 4 cp03
```

```
ping -c 4 etcd01
```

```
ping -c 4 worker01
```

All pings should succeed.

SSH Connectivity

Passwordless SSH is optional but highly recommended for administration and automation.

Example:

```
ssh root@cp02
```

```
ssh root@worker01
```

If prompted for a password every time, you can later configure SSH keys.

> **Why?** Automation tools like Ansible rely on SSH key-based authentication.

Time Synchronization

Kubernetes certificates and etcd rely heavily on accurate system time.

Even a clock drift of a few minutes can cause TLS authentication failures.

Verify the current time:

```
timedatectl
```

Expected:

```
System clock synchronized: yes
NTP service: active
```

Verify IP Configuration

Run on every node:

```
ip addr
```

Confirm:

- Correct interface (e.g., `ens8`)
- Correct IP address
- Interface state is `UP`

Verify Routing

```
ip route
```

Expected:

```
default via 10.0.0.1 dev ens8
10.0.0.0/16 dev ens8
```

Firewall Planning

At this stage, ensure that required ports will be allowed between nodes.

We will configure services in later stages, but planning is essential now.

Examples:

Service	Port
Kubernetes API	6443
etcd Client	2379
etcd Peer	2380
kubelet	10250
Calico BGP (if used)	179
NodePort	30000–32767

> We will revisit firewall rules in the component installation stages.

Verification Checklist

Before moving to Stage 2, verify:

- ☐ All VMs created.
- ☐ Static IPs assigned.
- ☐ Hostnames configured.
- ☐ `/etc/hosts` updated.
- ☐ Nodes can ping each other.
- ☐ SSH connectivity works.
- ☐ Time synchronization is active.
- ☐ Default route is correct.
- ☐ Planned firewall rules documented.

Common Mistakes

Incorrect Hostnames

Duplicate hostnames can lead to certificate mismatches and node registration issues.

Incorrect IP Addresses

Using DHCP instead of static IPs may cause node identities to change after reboot.

Unsynchronized Time

TLS certificates depend on accurate clocks. Time drift often results in confusing authentication errors.

Network Isolation

If nodes cannot reach each other, Kubernetes components such as etcd and the API server will fail to communicate.

🧠 CKA Concepts Learned

By completing this stage, you should understand:

- Why Kubernetes requires stable infrastructure before installation.
- The importance of static IP addresses and consistent hostnames.
- The role of `/etc/hosts` during initial deployments.
- Why reliable networking and routing are prerequisites for cluster health.
- How accurate time synchronization affects TLS and etcd.
- Why direct node-to-node communication is essential in Kubernetes.

📌 Notes Based on *Our* Deployment

This section records decisions that are specific to **your cluster**:

- Infrastructure platform: **PICO Public Cloud**
- OS: **Ubuntu Server 24.04 LTS**
- Kubernetes version: **v1.31.14**
- Container runtime: **containerd**
- CNI: **Calico**
- External etcd: **3-node cluster**
- Control-plane endpoint (VIP): **10.0.0.100:6443**
- Pod CIDR: **192.168.0.0/16**
- Service CIDR: **10.96.0.0/12**
- Cluster name: **pico-prod**

Stage 2 – Preparing Every Kubernetes Node

Objective

Prepare every server that will become part of the Kubernetes cluster.

At the end of this stage every node will have:

- correct hostname
- required kernel modules
- required sysctl parameters
- swap disabled
- container runtime ready
- kubeadm
- kubelet
- kubectl
- CRI tools

No Kubernetes cluster exists yet.

We are only preparing Linux.

Nodes

Run this stage on

```
cp01
cp02
cp03

worker01
worker02
worker03
worker04
worker05
```

Do **NOT** run on

```
lb01
lb02
etcd01
etcd02
etcd03
```

because those machines have different purposes.

Why is this stage required?

Kubernetes itself is only software.

It depends on Linux features such as

- cgroups
- namespaces
- iptables
- bridges
- overlay filesystem
- networking

Ubuntu’s default configuration is not enough.

Therefore every Kubernetes node must be prepared before kubeadm can build a cluster.

Step 2.1 – Set Hostname

Why?

Every Kubernetes node must have a unique identity.

The hostname becomes the Kubernetes Node Name.

Example

```
kubectl get nodes
```

shows

```
cp01
cp02
worker01
```

Those names come directly from Linux hostnames.

Example on cp01

```
hostnamectl set-hostname cp01
```

Verify

```
hostnamectl
```

Expected

```
Static hostname: cp01
```

Repeat

```
cp02
cp03

worker01
worker02
worker03
worker04
worker05
```

Step 2.2 Configure /etc/hosts

Why?

Although DNS exists, Kubernetes installation becomes much easier if every node can resolve every other node.

Without this,

```
ping cp02
```

may fail.

Many administrators rely on DNS.

In labs and small production environments, `/etc/hosts` is simpler.

Edit

```
vim /etc/hosts
```

Example

```
10.0.0.11 cp01
10.0.0.12 cp02
10.0.0.13 cp03

10.0.0.21 etcd01
10.0.0.22 etcd02
10.0.0.23 etcd03

10.0.0.31 worker01
10.0.0.32 worker02
10.0.0.33 worker03
10.0.0.34 worker04
10.0.0.35 worker05

10.0.0.101 lb01
10.0.0.102 lb02
```

Verify

```
ping cp02
```

Expected

```
PING cp02 (10.0.0.12)
```

Step 2.3 Disable Swap

What is Swap?

Normally Linux uses RAM.

When RAM becomes full,

Linux writes memory pages to disk.

This is called Swap.

```
RAM
 ↓
Swap
```

Swap is much slower than RAM.

Why Kubernetes hates Swap

Imagine a Pod.

It requests

```
1 GB RAM
```

Kubernetes assumes that memory is available.

But Linux silently moves half of that memory into swap.

Now the Pod becomes extremely slow.

Kubernetes can no longer correctly schedule workloads.

Therefore kubelet refuses to run with swap enabled.

Check

```
swapon --show
```

If anything appears,

disable it.

```
swapoff -a
```

Permanent

```
vim /etc/fstab
```

Comment swap

```
#/swap.img none swap sw 0 0
```

Verify

```
swapon --show
```

Expected

(no output)

Step 2.4 Load Required Kernel Modules

What is a Kernel Module?

Linux kernel is modular.

Drivers and networking features can be loaded only when needed.

Kubernetes requires two important modules.

overlay

Provides OverlayFS.

Container images work using layered filesystems.

Example

```
Ubuntu Image
      +
Nginx Layer
      +
Your Files
```

OverlayFS combines all layers.

Without it,

containerd cannot efficiently run containers.

---

br_netfilter

Linux bridges normally ignore iptables.

Kubernetes networking depends on iptables.

Calico

Flannel

Cilium

kube-proxy

all need packets traversing Linux bridges to be inspected by netfilter.

Without

```
br_netfilter
```

Pods may communicate incorrectly.

Services may fail.

Network policies may never work.

Create

```
cat <<EOF >/etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
```

Load immediately

```
modprobe overlay
modprobe br_netfilter
```

Verify

```
lsmod | grep overlay

lsmod | grep br_netfilter
```

Expected

```
overlay
br_netfilter
```

Step 2.5 Configure Kernel Networking

Kubernetes requires Linux networking behavior different from Ubuntu defaults.

Create

```
cat <<EOF >/etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables=1
net.bridge.bridge-nf-call-ip6tables=1
net.ipv4.ip_forward=1
EOF
```

Apply

```
sysctl --system
```

What does each parameter mean?

---

net.ipv4.ip_forward=1

Allows Linux to route packets.

Without it

Worker A

↓

Worker B

communication fails.

Every Kubernetes node behaves like a router for Pods.

Therefore forwarding must be enabled.

Verify

```
sysctl net.ipv4.ip_forward
```

Expected

```
1
```

---

bridge-nf-call-iptables

Normally

```
Pod A

↓

Linux Bridge

↓

Pod B
```

packets bypass iptables.

Kubernetes Services

Calico

Network Policies

all require iptables inspection.

Setting

```
bridge-nf-call-iptables=1
```

forces packets through iptables.

Verify

```
sysctl net.bridge.bridge-nf-call-iptables
```

Expected

```
1
```

Step 2.6 Install Common Packages

Install utilities used throughout the deployment.

```
apt update

apt install -y \
curl \
wget \
vim \
git \
jq \
apt-transport-https \
ca-certificates \
gnupg \
software-properties-common
```

Why each package?

Package	Purpose
curl	Download manifests and APIs
wget	Download files
vim	Edit configuration
git	Clone repositories
jq	Parse JSON output from Kubernetes APIs
apt-transport-https	Access HTTPS repositories
ca-certificates	Validate TLS certificates
gnupg	Verify repository signing keys
software-properties-common	Manage repositories

Verification Checklist

Run on every Kubernetes node:

```
hostnamectl

swapon --show

lsmod | grep overlay

lsmod | grep br_netfilter

sysctl net.ipv4.ip_forward

sysctl net.bridge.bridge-nf-call-iptables
```

Expected:

- Correct hostname
- No swap
- `overlay` loaded
- `br_netfilter` loaded
- `ip_forward = 1`
- `bridge-nf-call-iptables = 1`

Common Mistakes

Forgot to disable swap

Result:

```
kubeadm init

ERROR Swap is enabled
```

Forgot `br_netfilter`

Symptoms:

- Services don’t work correctly.
- Calico networking issues.
- NetworkPolicy doesn’t behave as expected.

Forgot `ip_forward`

Symptoms:

Pods on different nodes cannot communicate.

Knowledge Check (CKA Style)

By the end of Stage 2, you should be able to answer:

- Why does Kubernetes require swap to be disabled?
- What problem does `overlay` solve?
- Why is `br_netfilter` required for kube-proxy and Calico?
- Why must every node enable `net.ipv4.ip_forward`?
- Why do we configure `/etc/hosts` even if DNS exists?

If you can answer these without looking at the notes, you've understood the foundation rather than just memorized commands.

The next stage (Stage 3) will cover **installing and configuring containerd**, including the Container Runtime Interface (CRI), cgroups, the pause image, why `SystemdCgroup=true` is important, and how kubelet communicates with the runtime. This is one of the most important concepts for both real-world Kubernetes administration and the CKA exam.

> **Editorial note — gap found during review:** Stage 2's objectives list `kubeadm`, `kubelet`, and `kubectl` as things every node will have by the end of that stage, but no install commands for those three packages actually appear anywhere in the source notes — the runbook jumps from OS prep straight to containerd, then straight to etcd. The apt-based install (added below) is standard upstream Kubernetes procedure for the packages, versions, and repository this deployment already commits to (`v1.31.14`, `containerd`), inserted here to close that gap.

Stage 2.9 — Install kubeadm, kubelet, and kubectl *(added during review)*

Run on every node (control plane, etcd is optional, and workers)

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | \
  gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.31/deb/ /' | \
  tee /etc/apt/sources.list.d/kubernetes.list

apt update

apt install -y kubelet=1.31.14-* kubeadm=1.31.14-* kubectl=1.31.14-*

apt-mark hold kubelet kubeadm kubectl
```

Why this matters

- The `pkgs.k8s.io` repo is versioned per minor release (`v1.31` here) — pin to the minor line that matches the `kubernetesVersion` used later in `kubeadm-config.yaml`.
- `apt-mark hold` prevents `apt upgrade` from silently bumping these packages; Kubernetes upgrades must be done deliberately, one minor version at a time, via `kubeadm upgrade`.
- `kubelet` starts and crash-loops until `kubeadm init` / `kubeadm join` gives it a configuration — that's expected at this point.
- This step must happen on etcd nodes only if they'll also run kubelet; in a pure external-etcd design, etcd nodes don't need these three packages at all.

Verify

```
kubeadm version
kubelet --version
kubectl version --client
```

Stage 3 – Install and Configure the Container Runtime (containerd)

Objective

Install and configure the container runtime that Kubernetes will use to create and manage containers.

At the end of this stage:

- containerd is installed
- containerd service is running
- CRI plugin is enabled
- SystemdCgroup is configured
- kubelet can communicate with containerd
- crictl can inspect containers

Nodes

Run this stage on:

```
cp01
cp02
cp03

worker01
worker02
worker03
worker04
worker05
```

Do **NOT** run on:

```
lb01
lb02

etcd01
etcd02
etcd03
```

Reason:

- Load balancers do not run Pods.
- External etcd nodes only run the etcd database.

What is a Container Runtime?

Kubernetes does **not** run containers itself.

Many beginners think this:

```
Kubernetes
      ↓
Runs Containers
```

This is **wrong**.

The actual architecture is:

```
               kube-apiserver
                     |
                 Scheduler
                     |
                  kubelet
                     |
       Container Runtime Interface (CRI)
                     |
                containerd
                     |
                   runc
                     |
             Linux Kernel
```

Kubernetes only decides:

- which Pod should run
- on which node
- when to restart it

The **container runtime** actually starts and stops containers.

Why containerd?

Originally Kubernetes used Docker.

```
Kubernetes
     ↓
Docker
     ↓
containerd
     ↓
runc
```

Docker contained many components Kubernetes never needed.

Since Kubernetes v1.24:

- Dockershim was removed.
- Kubernetes communicates directly with CRI-compatible runtimes.

Today the recommended runtime is:

- containerd ✅

Other supported runtimes:

- CRI-O
- Mirantis Container Runtime

What is CRI?

CRI means

Container Runtime Interface

Think of it as a translator.

```
kubelet

"Start this Pod"

↓

CRI

↓

containerd

↓

Container starts
```

Without CRI,

kubelet cannot communicate with containerd.

Step 3.1 Install containerd

Update package index:

```
apt update
```

Install:

```
apt install -y containerd
```

Why?

This installs:

- containerd daemon
- containerd service
- default configuration
- CRI plugin

Verify:

```
containerd --version
```

Example:

```
containerd github.com/containerd/containerd 1.7.x
```

Step 3.2 Generate the Default Configuration

Create the configuration directory:

```
mkdir -p /etc/containerd
```

Generate the default configuration:

```
containerd config default > /etc/containerd/config.toml
```

Why?

Without this file, containerd uses internal defaults.

Generating it allows us to customize settings such as:

- cgroups
- sandbox image
- snapshotter
- CRI plugin

Verify:

```
ls -lh /etc/containerd/config.toml
```

Step 3.3 Configure Systemd Cgroups

Open:

```
vim /etc/containerd/config.toml
```

Find:

```
SystemdCgroup = false
```

Change it to:

```
SystemdCgroup = true
```

Why is this important?

This is one of the most common interview and CKA questions.

Linux manages resources through **cgroups** (control groups).

Both kubelet and containerd need to agree on **who manages cgroups**.

Wrong configuration

```
kubelet → systemd

containerd → cgroupfs
```

Now two different managers try to control the same container.

This causes:

- kubelet instability
- node registration failures
- Pods restarting
- resource accounting problems

Correct configuration

```
kubelet
      |
      ▼
systemd
      |
      ▼
containerd
```

Both use the same cgroup manager.

This is the Kubernetes recommendation.

Verify:

```
grep SystemdCgroup /etc/containerd/config.toml
```

Expected:

```
SystemdCgroup = true
```

Step 3.4 Restart containerd

Reload the service:

```
systemctl restart containerd
```

Enable it at boot:

```
systemctl enable containerd
```

Verify:

```
systemctl status containerd --no-pager
```

Expected:

```
Active: active (running)
```

Step 3.5 Verify the CRI Plugin

Check:

```
ctr plugins ls | grep cri
```

Expected:

```
io.containerd.grpc.v1 cri ok
```

Why?

If CRI is missing:

kubelet cannot communicate with containerd.

The cluster will never start.

Step 3.6 Install crictl

What is crictl?

`crictl` is like `kubectl`, but for the container runtime.

Think of it like this:

```
kubectl

↓

Kubernetes API
```

```
crictl

↓

containerd directly
```

If Kubernetes isn't working, `crictl` is the first troubleshooting tool to use.

Install:

```
VERSION="v1.31.1"

wget https://github.com/kubernetes-sigs/cri-tools/releases/download/${VERSION}/crictl-${VERSION}-linux-amd64.tar.gz

tar -C /usr/local/bin -xzf crictl-${VERSION}-linux-amd64.tar.gz

rm crictl-${VERSION}-linux-amd64.tar.gz
```

Verify:

```
crictl --version
```

Step 3.7 Configure crictl

Create:

```
cat <<EOF >/etc/crictl.yaml
runtime-endpoint: unix:///run/containerd/containerd.sock
image-endpoint: unix:///run/containerd/containerd.sock
timeout: 10
debug: false
EOF
```

Why?

Without this file,

`crictl` doesn't know which runtime to connect to.

Verify:

```
crictl info
```

Expected:

A large JSON output showing runtime information.

Step 3.8 Verify the Runtime Socket

Check:

```
ls -l /run/containerd/containerd.sock
```

Expected:

```
srw-rw---- root root
```

Why?

This Unix socket is the communication channel between kubelet and containerd.

```
kubelet
    |
    ▼
/run/containerd/containerd.sock
    |
    ▼
containerd
```

If the socket is missing:

- kubelet cannot start Pods
- `kubeadm init` fails

- worker nodes cannot join

Verification Checklist

Run:

```
containerd --version

systemctl status containerd --no-pager

grep SystemdCgroup /etc/containerd/config.toml

ctr plugins ls | grep cri

crictl info

ls -l /run/containerd/containerd.sock
```

Everything should succeed.

Common Mistakes

Forgot `SystemdCgroup=true`

Symptoms:

- kubelet repeatedly restarts
- node stays `NotReady`
- kubeadm preflight warnings

CRI plugin disabled

Symptoms:

```
kubelet cannot connect to runtime
```

Wrong socket

Example:

```
runtime-endpoint:
unix:///var/run/containerd.sock
```

instead of

```
unix:///run/containerd/containerd.sock
```

Result:

```
crictl:
connection refused
```

Forgot to restart containerd

You changed `config.toml`, but the old configuration is still loaded.

Always restart:

```
systemctl restart containerd
```

Knowledge Check (CKA Style)

By the end of this stage, you should be able to answer:

1. Why does Kubernetes need a container runtime?
2. What is the role of the Container Runtime Interface (CRI)?
3. Why is `containerd` preferred over Docker in modern Kubernetes?
4. Why must `SystemdCgroup` be set to `true`?
5. What is the purpose of `crictl`, and when would you use it instead of `kubectl`?
6. What is the significance of `/run/containerd/containerd.sock`?

If you can answer these confidently, you've understood not just the commands, but the architecture behind them.

After Stage 3, the next stage will prepare the Kubernetes packages themselves (`kubeadm` , `kubelet` , and `kubectl`), explain their distinct roles, configure the Kubernetes repository, and get the nodes ready for cluster initialization. That naturally leads into the control plane bootstrap with `kubeadm init` and the HA setup. # Stage 4 — Deploy External etcd Cluster (Manual Installation)

Goal

Before Kubernetes can run, it needs a reliable datastore.

Instead of running etcd inside Kubernetes (Stacked etcd), we deployed an **external three-node etcd cluster**.

Our architecture:

```
etcd01
10.0.0.21

etcd02
10.0.0.22

etcd03
10.0.0.23
```

Later every API Server will communicate with these three nodes.

```
API Server
      |
      |
      ▼
+------------------------------+
| etcd01                       |
| etcd02                       |
| etcd03                       |
+------------------------------+
```

Why External etcd?

Production clusters usually separate Kubernetes from etcd.

Advantages

- Kubernetes upgrades don’t touch etcd
- Easier backup
- Better HA
- Easier disaster recovery
- Recommended by Kubernetes for production

Stage 4.1 Install etcd

Run on

```
etcd01
etcd02
etcd03
```

Example

```
mkdir -p /root/k8s-ha-deployment/03-etcd
```

Create directories

```
mkdir -p \
/etc/etcd \
/var/lib/etcd \
/root/k8s-ha-deployment/03-etcd/{ca,certs,templates,scripts}
```

Verify

```
tree /root/k8s-ha-deployment/03-etcd
```

Expected

```
03-etcd
├── ca
├── certs
├── templates
└── scripts
```

Theory

We keep all deployment files inside one workspace.

```
03-etcd/
```

contains everything required to recreate the cluster later.

Nothing inside here is used directly by etcd.

Instead,

later we copy certificates into

```
/etc/etcd
```

This separation makes rebuilding easier.

Stage 4.2 Create the etcd Certificate Authority

Run ONLY on

```
etcd01
```

Create CA configuration

```
vim /root/k8s-ha-deployment/03-etcd/ca/ca-config.json
```

Example

```
{
  "signing": {
    "default": {
      "expiry": "87600h"
    },
    "profiles": {
      "server": {
        "expiry": "87600h",
        "usages": [
          "signing",
          "key encipherment",
          "server auth",
          "client auth"
        ]
      },
      "peer": {
        "expiry": "87600h",
        "usages": [
          "signing",
          "key encipherment",
          "server auth",
          "client auth"
        ]
      },
      "client": {
        "expiry": "87600h",
        "usages": [
          "signing",
          "key encipherment",
          "client auth"
        ]
      }
    }
  }
}
```

Create CA CSR

```
vim /root/k8s-ha-deployment/03-etcd/ca/ca-csr.json
```

Example

```
{
  "CN": "etcd",
  "key": {
    "algo": "rsa",
    "size": 4096
  },
  "names": [
    {
      "C": "BD",
      "ST": "Dhaka",
      "L": "Dhaka",
      "O": "PICO",
      "OU": "Kubernetes"
    }
  ]
}
```

Generate CA

```
cd /root/k8s-ha-deployment/03-etcd/ca

cfssl gencert -initca ca-csr.json | cfssljson -bare ca
```

Verify

```
openssl x509 -in ca.pem -noout -subject -issuer
```

Expected

```
subject=...
issuer=...
```

Since it’s a CA,

Subject = Issuer.

Theory

The CA is the highest authority.

Every certificate in the etcd cluster will be signed by this CA.

Without this,

No TLS

No authentication

No encrypted communication

Stage 4.3 Generate Server Certificates

Run ONLY on

```
etcd01
```

For every etcd node create a CSR.

Example

```
server-etcd01.json
```

```
{
  "CN": "etcd",
  "hosts": [
    "10.0.0.21",
    "127.0.0.1"
  ],
  "key": {
    "algo": "rsa",
    "size": 4096
  },
  "names": [
    {
      "O": "PICO"
    }
  ]
}
```

Generate certificate

```
cd /root/k8s-ha-deployment/03-etcd/certs

cfssl gencert \
-ca=../ca/ca.pem \
-ca-key=../ca/ca-key.pem \
-config=../ca/ca-config.json \
-profile=server \
../templates/server-etcd01.json \
| cfssljson -bare server-etcd01
```

Repeat for

```
server-etcd02

server-etcd03
```

Theory

These certificates identify each etcd server.

When Kubernetes connects to etcd,

the API Server verifies

> “Am I really talking to etcd01?”

using this certificate.

Without it,

TLS verification fails.

Stage 4.4 Generate Peer Certificates

Run ONLY on

```
etcd01
```

Generate

```
peer-etcd01

peer-etcd02

peer-etcd03
```

using

```
-profile=peer
```

Theory

These certificates are **not** for Kubernetes.

They are used **only between etcd members**.

```
etcd01
     ↕
etcd02
     ↕
etcd03
```

Each node authenticates every other node before exchanging Raft log entries.

Without peer certificates,

the etcd cluster cannot form securely.

Stage 4.5 Generate Health Check Client Certificate

Run on

```
etcd01
```

Create

```
healthcheck-client.json
```

Generate

```
cfssl gencert \
-ca=../ca/ca.pem \
-ca-key=../ca/ca-key.pem \
-config=../ca/ca-config.json \
-profile=client \
../templates/healthcheck-client.json \
| cfssljson -bare healthcheck-client
```

Theory

This certificate is used only by

```
etcdctl
```

to perform health checks securely.

Example:

```
etcdctl endpoint health
```

Without it,

the command would fail because client authentication is required.

Stage 4.6 Generate API Server Client Certificate

Run on

```
etcd01
```

Create

```
apiserver-etcd-client.json
```

Generate

```
cd /root/k8s-ha-deployment/03-etcd/certs

cfssl gencert \
-ca=../ca/ca.pem \
-ca-key=../ca/ca-key.pem \
-config=../ca/ca-config.json \
-profile=client \
../templates/apiserver-etcd-client.json \
| cfssljson -bare apiserver-etcd-client
```

Verify

```
openssl x509 \
-in apiserver-etcd-client.pem \
-noout \
-subject
```

Theory

This is one of the most important certificates in the cluster.

Only the Kubernetes API Server uses it.

```
API Server
      |
      |  TLS Client Certificate
      ▼
etcd
```

When the API Server queries etcd, it presents this certificate as its identity.

The etcd server checks:

- Was this certificate signed by the trusted etcd CA?
- Is it valid?
- Is it intended for client authentication?

If all checks pass, etcd allows the API Server to read and write cluster state. If not, the connection is rejected.

Stage 4.7 Copy Certificates to Each etcd Node

Run from **etcd01** (or use `scp` from etcd01 to the other nodes).

Copy the appropriate certificates to each node:

For **etcd01**:

```
scp /root/k8s-ha-deployment/03-etcd/ca/ca.pem \
root@etcd01:/etc/etcd/ca.pem

scp /root/k8s-ha-deployment/03-etcd/certs/server-etcd01.pem \
root@etcd01:/etc/etcd/server.pem

scp /root/k8s-ha-deployment/03-etcd/certs/server-etcd01-key.pem \
root@etcd01:/etc/etcd/server-key.pem

scp /root/k8s-ha-deployment/03-etcd/certs/peer-etcd01.pem \
root@etcd01:/etc/etcd/peer.pem

scp /root/k8s-ha-deployment/03-etcd/certs/peer-etcd01-key.pem \
root@etcd01:/etc/etcd/peer-key.pem
```

Repeat the same pattern for `etcd02` and `etcd03`, using their respective server and peer certificates.

> **Note:** Do **not** copy `apiserver-etcd-client.pem` to the etcd nodes. That certificate belongs to the Kubernetes control plane and will later be copied to the control-plane nodes.

In the next section (Stage 4.8), we'll configure the `etcd` systemd service, start the three-node cluster, and verify quorum using `etcdctl endpoint health` and `etcdctl endpoint status`. This is the point where the external etcd cluster becomes fully operational.

Now we move to the most important part of Stage 4. Up until now we only created certificates. The etcd cluster still doesn't exist. In this stage we'll configure and start the cluster.

Stage 4.8 – Configure the External etcd Cluster

Goal

At this point we have:

- ✅ Installed etcd binaries
- ✅ Created the etcd CA
- ✅ Generated all server certificates
- ✅ Generated all peer certificates
- ✅ Generated the healthcheck client certificate
- ✅ Generated the API Server client certificate

Now we need to tell each etcd node:

- What its name is
- Where to store its database
- Which certificate to use
- Which peers belong to the cluster
- How to communicate securely

Only after this step does the three-node etcd cluster become operational.

Theory

Think of etcd as three database servers that continuously synchronize with each other.

```
          Raft Consensus

      +-----------+
      | etcd01    |
      +-----------+
        ↑       ↓
        |       |
+-----------+  +-----------+
| etcd02    |  | etcd03    |
+-----------+  +-----------+
```

Each node must know:

- Who am I?
- Where are my friends?
- Which certificate should I trust?
- Which certificate should I present?

If any of these are wrong, the cluster will never form.

Files Used

Run on

```
etcd01
etcd02
etcd03
```

Create

```
/etc/etcd/etcd.conf.yml
```

Each node has **its own configuration**.

etcd01 Configuration

Run on

```
etcd01
```

```
name: etcd01

data-dir: /var/lib/etcd

listen-peer-urls: https://10.0.0.21:2380

listen-client-urls: https://10.0.0.21:2379,https://127.0.0.1:2379

initial-advertise-peer-urls: https://10.0.0.21:2380

advertise-client-urls: https://10.0.0.21:2379

initial-cluster-token: pico-etcd

initial-cluster: >
  etcd01=https://10.0.0.21:2380,
  etcd02=https://10.0.0.22:2380,
  etcd03=https://10.0.0.23:2380

initial-cluster-state: new

client-transport-security:
  cert-file: /etc/etcd/server.pem
  key-file: /etc/etcd/server-key.pem
  client-cert-auth: true
  trusted-ca-file: /etc/etcd/ca.pem

peer-transport-security:
  cert-file: /etc/etcd/peer.pem
  key-file: /etc/etcd/peer-key.pem
  peer-client-cert-auth: true
  trusted-ca-file: /etc/etcd/ca.pem
```

etcd02 Configuration

Run on

```
etcd02
```

Exactly the same except

```
name: etcd02

listen-peer-urls:
https://10.0.0.22:2380

listen-client-urls:
https://10.0.0.22:2379

advertise-client-urls:
https://10.0.0.22:2379

initial-advertise-peer-urls:
https://10.0.0.22:2380
```

Everything else remains identical.

etcd03 Configuration

Run on

```
etcd03
```

Only change

```
name: etcd03

listen-peer-urls:
https://10.0.0.23:2380

listen-client-urls:
https://10.0.0.23:2379

advertise-client-urls:
https://10.0.0.23:2379

initial-advertise-peer-urls:
https://10.0.0.23:2380
```

Understanding Every Parameter

name

```
name: etcd01
```

Every member must have a unique identity.

Think of it like the hostname inside the cluster.

data-dir

```
data-dir: /var/lib/etcd
```

This is where the entire Kubernetes database is stored.

Everything eventually lives here.

Examples:

- Pods
- Deployments
- Secrets
- ConfigMaps
- Nodes
- RBAC
- Events

If this directory is lost, the cluster state is lost unless restored from a snapshot.

listen-peer-urls

```
listen-peer-urls:
https://10.0.0.21:2380
```

Port **2380** is only used between etcd nodes.

```
etcd01
   │2380
   ▼
etcd02
```

Workers never use this port.

The API Server never uses this port.

Only Raft replication uses port **2380**.

listen-client-urls

```
https://10.0.0.21:2379
```

Port **2379** accepts client connections.

Clients include:

- kube-apiserver
- etcdctl
- backup scripts

---

advertise-client-urls

This tells clients:

> "If someone wants to contact me, use this address."

Without it, the API Server could receive an unreachable address.

---

initial-cluster

```
initial-cluster:
```

This is simply the member list.

```
etcd01

etcd02

etcd03
```

Every node **must** have the same list.

---

initial-cluster-token

```
pico-etcd
```

A unique identifier for this cluster.

It prevents accidentally joining members from another cluster.

---

initial-cluster-state

```
new
```

Only used when creating a brand-new cluster.

If restoring from a snapshot later, this value changes.

---

client-transport-security

```
client-cert-auth: true
```

This tells etcd:

> "Only clients with valid certificates are allowed."

Without a certificate, access is denied.

---

peer-transport-security

Exactly the same idea, but for communication between etcd members.

```
etcd01
      TLS
        ↕
etcd02
```

No trusted certificate = no replication.

Verify the Configuration

Run on each node:

```
cat /etc/etcd/etcd.conf.yml
```

Check:

- Node name is correct.
- IP addresses match the node.
- Peer list contains all three members.
- Certificate paths exist.

Verify Certificates

Run on every etcd node:

```
ls -lh /etc/etcd
```

Expected files:

```
ca.pem

server.pem
server-key.pem

peer.pem
peer-key.pem
```

What Happens Next?

Once the configuration is complete on all three nodes, the next stage is to:

1. Create the `etcd.service` systemd unit.
2. Enable and start the etcd service on each node.
3. Verify cluster formation with `etcdctl endpoint health` and `etcdctl endpoint status`.
4. Confirm that all three members have formed a healthy quorum before moving on to Kubernetes control plane setup.

This is the point where the external etcd cluster transitions from configuration files to a functioning distributed database.

This is where the etcd cluster actually comes alive. After this stage, your three-node external etcd cluster will be running and ready for Kubernetes.

Stage 4.9 – Create and Start the etcd Service

Goal

Up to now we have:

- ✅ Installed etcd
- ✅ Generated the PKI
- ✅ Generated server and peer certificates
- ✅ Created `etcd.conf.yml`

Now we will:

- Create the `systemd` service
- Start etcd
- Form the 3-node cluster
- Verify quorum

Theory

`systemd` is Linux's service manager.

Instead of manually starting etcd every time with:

```
etcd
```

we create a permanent service.

Advantages:

- Starts automatically after reboot
- Automatically restarts if it crashes
- Easy management with `systemctl`
- Standard production practice

Step 1 – Create the systemd Service

Run on:

```
etcd01
etcd02
etcd03
```

Create:

```
vim /etc/systemd/system/etcd.service
```

Contents:

```
[Unit]
Description=etcd
Documentation=https://etcd.io
After=network.target
Wants=network-online.target

[Service]
Type=notify

ExecStart=/usr/local/bin/etcd \
  --config-file=/etc/etcd/etcd.conf.yml

Restart=always
RestartSec=5

LimitNOFILE=40000

[Install]
WantedBy=multi-user.target
```

Understanding Every Line

After=network.target

Wait until networking is available.

Otherwise etcd starts before the NIC receives its IP.

ExecStart

```
ExecStart=/usr/local/bin/etcd \
--config-file=/etc/etcd/etcd.conf.yml
```

This tells systemd:

> Start etcd and load all settings from our configuration file.

Notice that we don't pass 20 different command-line arguments. Everything is centralized in `etcd.conf.yml`, which is cleaner and easier to maintain.

Restart=always

If etcd crashes:

```
Crash

↓

Restart automatically

↓

Cluster stays available
```

Production clusters should always use this.

LimitNOFILE

```
40000
```

etcd can keep thousands of network connections open.

Linux normally limits this to around 1024.

Increasing it prevents “too many open files” errors in larger clusters.

Step 2 – Reload systemd

Run on all three nodes:

```
systemctl daemon-reload
```

This tells systemd to read the newly created service file.

Step 3 – Enable etcd

Run on all three nodes:

```
systemctl enable etcd
```

Why?

Without this, etcd would not start automatically after a reboot.

Step 4 – Start etcd

Run on all three nodes:

```
systemctl start etcd
```

Step 5 – Verify Service Status

Run on each node:

```
systemctl status etcd --no-pager
```

Expected:

```
Active: active (running)
```

Step 6 – Check Logs

If there are any issues:

```
journalctl -u etcd -f
```

or

```
journalctl -u etcd --no-pager -n 50
```

Understanding the Logs

A healthy startup typically includes messages indicating:

- Member started
- Published local member
- Established peer connections
- Ready to serve client requests

If instead you see:

- certificate errors
- x509 validation failures
- connection refused
- failed to publish local member

the cluster has not formed correctly.

Step 7 – Verify Cluster Health

Run **only on** `etcd01` (or any etcd node with the client certificates):

```
export ETCDCTL_API=3

etcdctl \
  --cacert=/root/k8s-ha-deployment/03-etcd/ca/ca.pem \
  --cert=/root/k8s-ha-deployment/03-etcd/certs/healthcheck-client.pem \
  --key=/root/k8s-ha-deployment/03-etcd/certs/healthcheck-client-key.pem \
  --endpoints=https://10.0.0.21:2379,https://10.0.0.22:2379,https://10.0.0.23:2379 \
  endpoint health
```

Why use the healthcheck client certificate?

Earlier in Stage 4 we created a dedicated client certificate for `etcdctl`. Since client certificate authentication is enabled, `etcdctl` must authenticate itself before etcd will respond.

Expected Output

```
https://10.0.0.21:2379 is healthy

https://10.0.0.22:2379 is healthy

https://10.0.0.23:2379 is healthy
```

Step 8 – Verify Member Status

Run:

```
export ETCDCTL_API=3

etcdctl \
  --cacert=/root/k8s-ha-deployment/03-etcd/ca/ca.pem \
  --cert=/root/k8s-ha-deployment/03-etcd/certs/healthcheck-client.pem \
  --key=/root/k8s-ha-deployment/03-etcd/certs/healthcheck-client-key.pem \
  --endpoints=https://10.0.0.21:2379 \
  member list
```

Expected:

```
etcd01

etcd02

etcd03
```

All three members should appear.

Step 9 – Check Detailed Endpoint Status

Run:

```
export ETCDCTL_API=3

etcdctl \
  --cacert=/root/k8s-ha-deployment/03-etcd/ca/ca.pem \
  --cert=/root/k8s-ha-deployment/03-etcd/certs/healthcheck-client.pem \
  --key=/root/k8s-ha-deployment/03-etcd/certs/healthcheck-client-key.pem \
  --endpoints=https://10.0.0.21:2379,https://10.0.0.22:2379,https://10.0.0.23:2379 \
  endpoint status --write-out=table
```

Expected table (example):

ENDPOINT	ID	VERSION	DB SIZE	IS LEADER
etcd01	...	3.5.x	...	true/false
etcd02	...	3.5.x	...	true/false
etcd03	...	3.5.x	...	true/false

Exactly **one** member should have `IS LEADER = true`.

What Happens Internally?

After all three services start:



The nodes elect a leader using the **Raft consensus algorithm**.

- One leader accepts writes.
- The followers replicate the leader's log.
- A majority (2 out of 3) is required for writes.
- If the leader fails, the remaining two nodes automatically elect a new leader.

This is why a three-node etcd cluster can tolerate the loss of one member without losing availability.

Stage 4 Completion Checklist

Before moving to Stage 5 (Kubernetes PKI), verify:

- ✅ `etcd.service` exists on all three nodes.
- ✅ `systemctl status etcd` shows `active (running)` on all nodes.
- ✅ `etcdctl endpoint health` reports all three endpoints as healthy.
- ✅ `etcdctl member list` shows three members.
- ✅ `etcdctl endpoint status --write-out=table` shows one leader and two followers.

At this point, your external etcd cluster is fully operational and ready to be used by the Kubernetes API servers in the next stage.

Now we reach **Stage 5**, which is one of the most important stages of the entire deployment.

This stage is where we **manually build Kubernetes PKI**. This is **not** required when using a normal `kubeadm init`, but we intentionally did it to understand Kubernetes internals before letting kubeadm consume these certificates.

Stage 5 – Kubernetes PKI (Manual Certificate Generation)

Goal

In Stage 4, we secured **etcd**.

Now we need to secure **Kubernetes itself**.

Every Kubernetes component communicates over TLS.

Examples:

```
kubectl
      |
      | TLS
      ▼
kube-apiserver
      |
      | TLS
      ▼
etcd

kube-controller-manager
      |
      ▼
kube-apiserver

kube-scheduler
      |
      ▼
kube-apiserver

kubelet
      |
      ▼
kube-apiserver
```

Without certificates:

- Components cannot authenticate each other.
- Connections are rejected.
- The cluster cannot function.

Stage Overview

In this stage we generate:

Certificate	Used By	Purpose
Kubernetes CA	Entire Cluster	Root CA
kube-apiserver	API Server	Server certificate
admin	kubectl	Cluster administrator
kube-controller-manager	Controller Manager	Client certificate
kube-scheduler	Scheduler	Client certificate
kube-proxy	kube-proxy	Client certificate
Service Account Key	API Server	Sign Service Account tokens

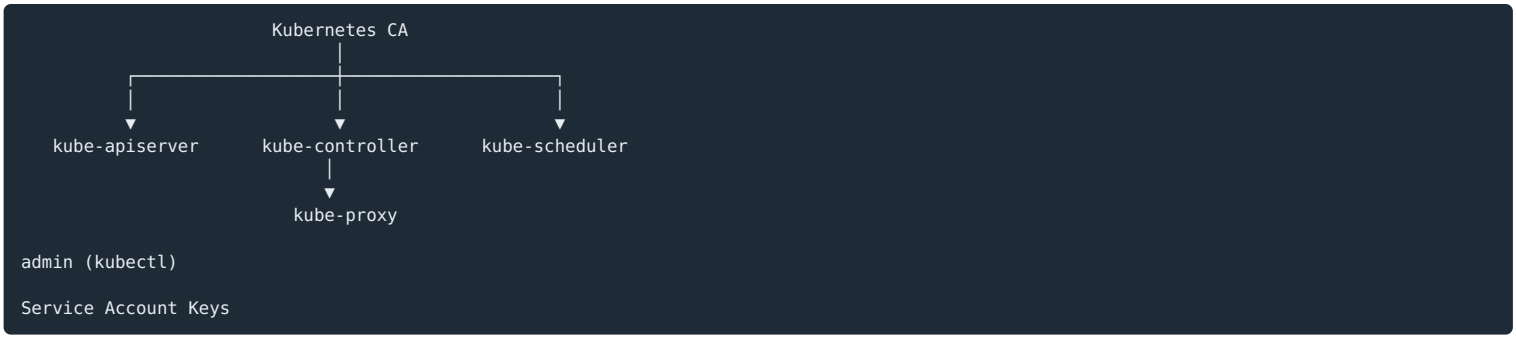
Why Are We Doing This Manually?

Most people never see these certificates because `kubeadm` generates them automatically.

We generated them manually because:

- Learn Kubernetes internals.
- Understand PKI.
- Troubleshoot TLS problems confidently.
- Know exactly which component uses which certificate.
- Build production-level understanding (valuable for CKA and real-world operations).

Cluster Architecture at This Stage



Every certificate above is signed by **one Kubernetes CA**.

Where We Performed This Stage

Unlike Stage 4, **all PKI generation was done only on `cp01`**.

Node:

```
cp01
```

Working directory:

```
/ root /k8s-ha-deployment/04-kubernetes
```

Structure:

```
04-kubernetes
│
├── ca
├── certs
├── templates
├── kubeconfig
├── configs
└── manifests
```

Stage 5 Breakdown

We completed the following sub-stages:

- **5.1** Create Kubernetes CA
- **5.2** Generate API Server Certificate
- **5.3** Generate Admin Certificate
- **5.4** Generate Controller Manager Certificate
- **5.5** Generate Scheduler Certificate
- **5.6** Generate kube-proxy Certificate
- **5.7** Generate Service Account Keys

Stage 5.1 – Create the Kubernetes Certificate Authority (CA)

Goal

Create the **root certificate authority** for Kubernetes.

This CA signs every Kubernetes certificate except the etcd certificates.

Run On

```
cp01
```

Working Directory

```
mkdir -p /root/k8s-ha-deployment/04-kubernetes/{ca,certs,templates,configs,kubeconfig,manifests,scripts}

cd /root/k8s-ha-deployment/04-kubernetes
```

Create CA Configuration

Create:

```
vim ca/ca-config.json
```

Example:

```
{
  "signing": {
    "default": {
      "expiry": "43800h"
    },
    "profiles": {
      "kubernetes": {
        "expiry": "43800h",
        "usages": [
          "signing",
          "key encipherment",
          "server auth",
          "client auth"
        ]
      }
    }
  }
}
```

Understanding This File

expiry

```
43800h
```

≈ 5 years

This means every certificate signed with the `kubernetes` profile will be valid for five years.

---

usages

```
signing
```

Allows the certificate to sign data.

```
key encipherment
```

Allows encrypted key exchange during TLS.

```
server auth
```

Can be used by servers.

Example:

```
kube-apiserver
```

```
client auth
```

Can be used by clients.

Example:

```
kubectl

scheduler

controller-manager
```

Create the CA CSR

```
vim ca/ca-csr.json
```

Example:

```
{
  "CN": "kubernetes",
  "key": {
    "algo": "rsa",
    "size": 4096
  },
  "names": [
    {
      "C": "BD",
      "ST": "Dhaka",
      "L": "Dhaka",
      "O": "PICO",
      "OU": "Kubernetes"
    }
  ]
}
```

Understanding the Fields

CN

```
Common Name
```

Represents the identity of the certificate.

Here:

```
kubernetes
```

means this certificate is the Kubernetes CA.

Organization

```
PICO
```

Used only for identification.

Key Size

```
4096-bit RSA
```

Provides strong security.

Generate the Kubernetes CA

Run:

```
cd /root/k8s-ha-deployment/04-kubernetes/ca

cfssl gencert -initca ca-csr.json | cfssljson -bare ca
```

Generated files:

```
ca.pem

ca-key.pem

ca.csr
```

Verify the Certificate

```
openssl x509 \
-in ca.pem \
-noout \
-subject \
-issuer \
-dates
```

Expected:

```
Subject = kubernetes

Issuer = kubernetes
```

Since it is self-signed:

```
Subject = Issuer
```

Verify the Private Key

```
openssl rsa \
-in ca-key.pem \
-check \
-noout
```

Expected:

```
RSA key ok
```

What Was Created?

```
ca.pem
```

Public certificate.

Every Kubernetes component trusts this certificate.

```
ca-key.pem
```

Private key.

Never expose this file.

Anyone possessing it can create trusted Kubernetes certificates.

Theory Recap

At this point, we have established the **root of trust** for the Kubernetes control plane.

Every certificate we generate in the remaining Stage 5 sub-stages (API Server, admin, controller manager, scheduler, kube-proxy) will be **signed by this CA**.

This mirrors how enterprises manage internal PKI: a trusted root CA signs the identities of every service and client.

Next Sections

We'll continue with:

- **Stage 5.2 – Generate the kube-apiserver certificate**
- **Stage 5.3 – Generate the admin certificate**
- **Stage 5.4 – Generate the kube-controller-manager certificate**
- **Stage 5.5 – Generate the kube-scheduler certificate**
- **Stage 5.6 – Generate the kube-proxy certificate**
- **Stage 5.7 – Generate the Service Account signing keys**

These will follow the same detailed format: **goal → commands → theory → verification → component usage**, matching exactly what we built during your deployment.

We'll continue with **Stage 5** exactly according to the deployment we performed.

Stage 5.2 — Generate the Kubernetes API Server Certificate

Goal

The API Server is the front door of Kubernetes.

Every component communicates with the API Server over HTTPS.

Therefore the API Server requires a TLS certificate.

Unlike the etcd certificates, this certificate contains many **Subject Alternative Names (SANs)** because different components connect to the API Server using different addresses.

Why is this certificate required?

Clients may access the API Server using:

- Kubernetes Service IP
- API VIP

- Control Plane IPs
- localhost

If those addresses are not included inside the certificate, TLS verification will fail.

Our Environment

Item	Value
Cluster Name	pico-prod
VIP	10.0.0.100
Service CIDR	10.96.0.0/12
Kubernetes Service IP	10.96.0.1
cp01	10.0.0.11
cp02	10.0.0.12
cp03	10.0.0.13

Run On

cp01 only

Step 1 — Create CSR Template

```
mkdir -p /root/k8s-ha-deployment/04-kubernetes/templates

cd /root/k8s-ha-deployment/04-kubernetes/templates

vim apiserver.json
```

Paste:

```
{
  "CN": "kube-apiserver",
  "hosts": [
    "10.96.0.1",
    "10.0.0.100",
    "10.0.0.11",
    "10.0.0.12",
    "10.0.0.13",
    "127.0.0.1",
    "kubernetes",
    "kubernetes.default",
    "kubernetes.default.svc",
    "kubernetes.default.svc.cluster.local"
  ],
  "key": {
    "algo": "rsa",
    "size": 4096
  },
  "names": [
    {
      "C": "BD",
      "ST": "Dhaka",
      "L": "Dhaka",
      "O": "Kubernetes",
      "OU": "Kubernetes"
    }
  ]
}
```

Why are these SANs included?

SAN	Reason
10.96.0.1	Kubernetes Service
10.0.0.100	HAProxy VIP
10.0.0.11	cp01
10.0.0.12	cp02
10.0.0.13	cp03
127.0.0.1	Local access
kubernetes	Internal DNS

kubernetes.default	Internal DNS
kubernetes.default.svc	Internal DNS
kubernetes.default.svc.cluster.local	Full Kubernetes DNS

Step 2 — Generate Certificate

```
cd /root/k8s-ha-deployment/04-kubernetes/certs

cfssl gencert \
-ca=../ca/ca.pem \
-ca-key=../ca/ca-key.pem \
-config=../ca/ca-config.json \
-profile=kubernetes \
../templates/apiserver.json | cfssljson -bare kube-apiserver
```

Generated Files

```
kube-apiserver.pem
kube-apiserver-key.pem
kube-apiserver.csr
```

Verify

```
openssl x509 \
-in kube-apiserver.pem \
-noout \
-text | grep -A10 "Subject Alternative Name"
```

Expected output includes:

```
10.96.0.1

10.0.0.100

10.0.0.11

10.0.0.12

10.0.0.13

127.0.0.1

kubernetes.default.svc.cluster.local
```

Concept Summary

The API Server certificate is the **server identity** of Kubernetes.

Every client verifies this certificate before trusting the API Server.

Without the correct SANs, all clients would reject the connection.

Stage 5.3 — Generate the Administrator Certificate

Goal

Create the administrator identity used by `kubectl`.

Why?

When you run:

```
kubectl get nodes
```

you are authenticating as a Kubernetes user.

That user is **admin**.

This certificate proves your identity to the API Server.

Run On

cp01 only

Step 1

```
cd /root/k8s-ha-deployment/04-kubernetes/templates

vim admin.json
```

Paste

```
{
  "CN": "admin",
  "key": {
    "algo": "rsa",
    "size": 4096
  },
  "names": [
    {
      "C": "BD",
      "ST": "Dhaka",
      "L": "Dhaka",
      "O": "system:masters",
      "OU": "Kubernetes"
    }
  ]
}
```

Why `system:masters`?

The `system:masters` group is bound to the `cluster-admin` ClusterRole by default.

This gives the certificate full administrative privileges.

Step 2

Generate:

```
cd /root/k8s-ha-deployment/04-kubernetes/certs

cfssl gencert \
-ca=../ca/ca.pem \
-ca-key=../ca/ca-key.pem \
-config=../ca/ca-config.json \
-profile=kubernetes \
../templates/admin.json | cfssljson -bare admin
```

Verify

```
openssl x509 \
-in admin.pem \
-noout \
-subject
```

Expected:

```
CN=admin

O=system:masters
```

Generated Files

```
admin.pem

admin-key.pem

admin.csr
```

Concept Summary

This certificate is equivalent to a Kubernetes root administrator.

Every `kubectl` command later uses this certificate.

Stage 5.4 — Generate the Controller Manager Certificate

Goal

Create the identity for the Kubernetes Controller Manager.

Why?

The Controller Manager is responsible for:

- Deployments
- ReplicaSets
- Jobs
- DaemonSets
- Endpoints
- ServiceAccounts
- Node Controller
- Namespace Controller

It continuously watches the API Server and ensures the cluster reaches the desired state.

Run On

cp01 only

Step 1

```
cd /root/k8s-ha-deployment/04-kubernetes/templates

vim kube-controller-manager.json
```

Paste

```
{
  "CN": "system:kube-controller-manager",
  "key": {
    "algo": "rsa",
    "size": 4096
  },
  "names": [
    {
      "C": "BD",
      "ST": "Dhaka",
      "L": "Dhaka",
      "O": "system:kube-controller-manager",
      "OU": "Kubernetes"
    }
  ]
}
```

Step 2

Generate

```
cd /root/k8s-ha-deployment/04-kubernetes/certs

cfssl gencert \
-ca=../ca/ca.pem \
-ca-key=../ca/ca-key.pem \
-config=../ca/ca-config.json \
-profile=kubernetes \
../templates/kube-controller-manager.json | cfssljson -bare kube-controller-manager
```

Verify

```
openssl x509 \
-in kube-controller-manager.pem \
-noout \
-subject
```

Expected

```
CN=system:kube-controller-manager

O=system:kube-controller-manager
```

Generated Files

```
kube-controller-manager.pem

kube-controller-manager-key.pem
```

Concept Summary

The Controller Manager authenticates to the API Server using this certificate.

Without it, the controller cannot manage cluster resources.

Stage 5.5 — Generate the Scheduler Certificate

Goal

Create the identity used by the Kubernetes Scheduler.

Why?

The Scheduler decides **which node should run each Pod**.

It continuously watches for Pods without an assigned node and binds them to the most appropriate worker.

Run On

cp01 only

Step 1

```
cd /root/k8s-ha-deployment/04-kubernetes/templates

vim kube-scheduler.json
```

Paste

```
{
  "CN": "system:kube-scheduler",
  "key": {
    "algo": "rsa",
    "size": 4096
  },
  "names": [
    {
      "C": "BD",
      "ST": "Dhaka",
      "L": "Dhaka",
      "O": "system:kube-scheduler",
      "OU": "Kubernetes"
    }
  ]
}
```

Step 2

Generate

```
cd /root/k8s-ha-deployment/04-kubernetes/certs

cfssl gencert \
-ca=../ca/ca.pem \
-ca-key=../ca/ca-key.pem \
-config=../ca/ca-config.json \
-profile=kubernetes \
../templates/kube-scheduler.json | cfssljson -bare kube-scheduler
```

Verify

```
openssl x509 \
-in kube-scheduler.pem \
-noout \
-subject
```

Expected

```
CN=system:kube-scheduler

O=system:kube-scheduler
```

Concept Summary

The Scheduler uses this certificate to securely communicate with the API Server and record scheduling decisions.

Stage 5.6 — Generate the Kube Proxy Certificate

Goal

Create the identity for `kube-proxy`.

Why?

`kube-proxy` runs on every node and programs iptables/IPVS rules so Services can forward traffic to Pods.

It must authenticate to the API Server to watch Services and Endpoints.

Run On

cp01 only

Step 1

```
cd /root/k8s-ha-deployment/04-kubernetes/templates

vim kube-proxy.json
```

Paste

```
{
  "CN": "system:kube-proxy",
  "key": {
    "algo": "rsa",
    "size": 4096
  },
  "names": [
    {
      "C": "BD",
      "ST": "Dhaka",
      "L": "Dhaka",
      "O": "system:node-proxier",
      "OU": "Kubernetes"
    }
  ]
}
```

Step 2

Generate

```
cd /root/k8s-ha-deployment/04-kubernetes/certs

cfssl gencert \
-ca=../ca/ca.pem \
-ca-key=../ca/ca-key.pem \
-config=../ca/ca-config.json \
-profile=kubernetes \
../templates/kube-proxy.json | cfssljson -bare kube-proxy
```

Verify

```
openssl x509 \
-in kube-proxy.pem \
-noout \
-subject
```

Expected

```
CN=system:kube-proxy

O=system:node-proxier
```

Concept Summary

This certificate allows every `kube-proxy` instance to securely watch networking objects from the API Server and keep Service networking functional.

Stage 5.7 — Generate the Service Account Key Pair

Goal

Generate the signing key pair used for Kubernetes Service Accounts.

Why?

Pods use **Service Account tokens** to authenticate with the API Server.

The API Server signs these tokens with the private key (`sa.key`), and other components verify them with the public key (`sa.pub`).

Run On

cp01 only

Step 1 — Generate the Private Key

```
cd /root/k8s-ha-deployment/04-kubernetes/certs

openssl genrsa -out sa-key.pem 4096
```

Step 2 — Extract the Public Key

```
openssl rsa \
-in sa-key.pem \
-pubout \
-out sa.pem
```

Generated Files

```
sa-key.pem   # Private signing key
sa.pem       # Public verification key
```

Verify the Private Key

```
openssl rsa \
-in sa-key.pem \
-check \
-noout
```

Expected output:

```
RSA key ok
```

Concept Summary

Unlike the previous certificates, these are **not TLS certificates**. They are an RSA key pair used specifically for **signing and verifying Service Account tokens**. Every Pod that uses a Service Account ultimately relies on these keys for secure authentication within the cluster.

At the end of **Stage 5**, you have all the cryptographic material required to bootstrap a production-grade Kubernetes control plane:

- Kubernetes CA
- API Server certificate
- Administrator certificate
- Controller Manager certificate
- Scheduler certificate
- Kube Proxy certificate
- Service Account signing keys

The next stage (Stage 6) uses these certificates to build kubeconfig files and the encryption configuration that Kubernetes components will use to communicate securely.

Stage 6 is where all the certificates you generated in Stage 5 become usable. This stage often confuses people, so I'll explain not just the commands, but *why* each kubeconfig is needed and *which node* it belongs to.

Stage 6 — Generate kubeconfig Files

Objective

In Stage 5 we created the identities (certificates).

Now we create the **configuration files** that Kubernetes components will use to connect to the API Server.

Think of it this way:

```
Certificate
    ↓
Who am I?

kubeconfig
    ↓
Where do I connect?
Which certificate do I use?
Which CA do I trust?
```

A kubeconfig combines four things:

- API Server endpoint
- Cluster CA certificate
- Client certificate
- Client private key

Without a kubeconfig, a component has no idea how to contact Kubernetes.

Components that need kubeconfig

Component	Needs kubeconfig?	Created On	Used On
kubectl (admin)	✍	cp01	All control planes
kube-controller-manager	✍	cp01	All control planes
kube-scheduler	✍	cp01	All control planes
kube-proxy	✍	cp01	Every worker node

Why do we generate them only on cp01?

Because **cp01 is our certificate generation host (PKI host)**.

All certificates already exist there.

After creating the kubeconfigs:

- copy them to cp02
- copy them to cp03
- copy kube-proxy.kubeconfig to workers

API Endpoint Used

All kubeconfigs point to:

```
https://10.0.0.100:6443
```

NOT

```
10.0.0.11
```

Why?

Because clients must always connect through the **VIP**.

If cp01 dies, HAProxy automatically forwards requests to cp02 or cp03.

No kubeconfig ever needs changing.

Stage 6.1 — Admin kubeconfig

Purpose

Used by

```
kubectl
```

This is your administrator login.

Run On

cp01 only

Create

```
mkdir -p /root/k8s-ha-deployment/04-kubernetes/kubeconfig

cd /root/k8s-ha-deployment/04-kubernetes/kubeconfig
```

Cluster

```
kubectl config set-cluster pico-prod \
  --certificate-authority=../ca/ca.pem \
  --embed-certs=true \
  --server=https://10.0.0.100:6443 \
  --kubeconfig=admin.kubeconfig
```

User

```
kubectl config set-credentials admin \
  --client-certificate=../certs/admin.pem \
  --client-key=../certs/admin-key.pem \
  --embed-certs=true \
  --kubeconfig=admin.kubeconfig
```

Context

```
kubectl config set-context default \
  --cluster=pico-prod \
  --user=admin \
  --kubeconfig=admin.kubeconfig
```

Use Context

```
kubectl config use-context default \
  --kubeconfig=admin.kubeconfig
```

Verify

```
kubectl config view \
--kubeconfig=admin.kubeconfig
```

Expected

```
Cluster:
    pico-prod

Server:
    https://10.0.0.100:6443

User:
    admin
```

Theory

This file becomes

```
/etc/kubernetes/admin.conf
```

Later

```
cp admin.conf ~/.kube/config
```

allows

```
kubectl get nodes
```

to authenticate as the cluster administrator.

Stage 6.2 — Controller Manager kubeconfig

Purpose

Used by

```
kube-controller-manager
```

Cluster

```
kubectl config set-cluster pico-prod \
  --certificate-authority=../ca/ca.pem \
  --embed-certs=true \
  --server=https://10.0.0.100:6443 \
  --kubeconfig=kube-controller-manager.kubeconfig
```

Credentials

```
kubectl config set-credentials system:kube-controller-manager \
  --client-certificate=../certs/kube-controller-manager.pem \
  --client-key=../certs/kube-controller-manager-key.pem \
  --embed-certs=true \
  --kubeconfig=kube-controller-manager.kubeconfig
```

Context

```
kubectl config set-context default \
  --cluster=pico-prod \
  --user=system:kube-controller-manager \
  --kubeconfig=kube-controller-manager.kubeconfig
```

Use

```
kubectl config use-context default \
  --kubeconfig=kube-controller-manager.kubeconfig
```

Verify

```
kubectl config view \
--kubeconfig=kube-controller-manager.kubeconfig
```

Theory

The Controller Manager uses this kubeconfig to:

- watch Deployments
- manage ReplicaSets
- allocate ServiceAccounts
- monitor Nodes
- update Endpoints

Without it, the Controller Manager cannot communicate with the API Server.

Stage 6.3 — Scheduler kubeconfig

Purpose

Used by

```
kube-scheduler
```

Cluster

```
kubectl config set-cluster pico-prod \
  --certificate-authority=../ca/ca.pem \
  --embed-certs=true \
  --server=https://10.0.0.100:6443 \
  --kubeconfig=kube-scheduler.kubeconfig
```

Credentials

```
kubectl config set-credentials system:kube-scheduler \
  --client-certificate=../certs/kube-scheduler.pem \
  --client-key=../certs/kube-scheduler-key.pem \
  --embed-certs=true \
  --kubeconfig=kube-scheduler.kubeconfig
```

Context

```
kubectl config set-context default \
  --cluster=pico-prod \
  --user=system:kube-scheduler \
  --kubeconfig=kube-scheduler.kubeconfig
```

Use

```
kubectl config use-context default \
  --kubeconfig=kube-scheduler.kubeconfig
```

Verify

```
kubectl config view \
--kubeconfig=kube-scheduler.kubeconfig
```

Theory

Whenever a Pod has no assigned node:

```
Pending
```

the Scheduler uses this kubeconfig to:

- watch Pending Pods
- choose the best node
- bind the Pod to that node

Stage 6.4 — kube-proxy kubeconfig

Purpose

Used by

```
kube-proxy
```

on every node.

Cluster

```
kubectl config set-cluster pico-prod \
  --certificate-authority=../ca/ca.pem \
  --embed-certs=true \
  --server=https://10.0.0.100:6443 \
  --kubeconfig=kube-proxy.kubeconfig
```

Credentials

```
kubectl config set-credentials system:kube-proxy \
  --client-certificate=../certs/kube-proxy.pem \
  --client-key=../certs/kube-proxy-key.pem \
  --embed-certs=true \
  --kubeconfig=kube-proxy.kubeconfig
```

Context

```
kubectl config set-context default \
  --cluster=pico-prod \
  --user=system:kube-proxy \
  --kubeconfig=kube-proxy.kubeconfig
```

Use

```
kubectl config use-context default \
  --kubeconfig=kube-proxy.kubeconfig
```

Verify

```
kubectl config view \
--kubeconfig=kube-proxy.kubeconfig
```

Theory

`kube-proxy` continuously watches the API Server for:

- Services
- Endpoints
- EndpointSlices

It then creates iptables/IPVS rules so that traffic sent to a Service IP reaches the correct backend Pods.

Without this kubeconfig, Service networking would not function.

Stage 6.5 — Encryption Configuration

Purpose

Protect Kubernetes Secrets stored in etcd.

By default:

```
Secret
↓

API Server

↓

etcd
```

Without encryption, Secrets are stored as plain data (base64-encoded, but not encrypted).

With an EncryptionConfig:

```
Secret

↓

AES Encryption

↓

etcd
```

Even if someone gains direct access to etcd, they cannot read the Secret values without the encryption key.

Run On

cp01 only

Generate a random encryption key

```
head -c 32 /dev/urandom | base64
```

Example output:

```
6lbcIKRjnZV7lsWuMvGW74QHIcR+sK8SSs0QU/VZo=
```

Use your generated value in the configuration below.

Create the configuration

```
mkdir -p /root/k8s-ha-deployment/04-kubernetes/configs

cd /root/k8s-ha-deployment/04-kubernetes/configs

vim encryption-config.yaml
```

Paste:

```
kind: EncryptionConfig
apiVersion: apiserver.config.k8s.io/v1

resources:
- resources:
  - secrets

  providers:
  - aescbc:
      keys:
      - name: key1
        secret: <YOUR_BASE64_KEY>

  - identity: {}
```

Replace `<YOUR_BASE64_KEY>` with the output from the `head -c 32 /dev/urandom | base64` command.

Verify

```
cat encryption-config.yaml
```

You should see the `aescbc` provider with your generated key followed by the `identity` provider.

Stage 6 Summary

At the end of Stage 6, you should have:

```
04-kubernetes/
├── kubeconfig/
│   ├── admin.kubeconfig
│   ├── kube-controller-manager.kubeconfig
│   ├── kube-scheduler.kubeconfig
│   └── kube-proxy.kubeconfig
│
└── configs/
    └── encryption-config.yaml
```

What each file is for

File	Used By	Target Nodes
`admin.kubeconfig`	`kubectl` administrator	All control-plane nodes
`kube-controller-manager.kubeconfig`	Controller Manager	All control-plane nodes
`kube-scheduler.kubeconfig`	Scheduler	All control-plane nodes
`kube-proxy.kubeconfig`	kube-proxy	Every worker (and control-plane if running kube-proxy)
`encryption-config.yaml`	kube-apiserver	All control-plane nodes

In **Stage 7**, these files are copied into their final system locations, and we prepare the control-plane nodes for `kubeadm init` using the external etcd cluster and the certificates you've created. This is where the PKI and kubeconfig work you've done begins to power the actual cluster.

This is the point where our deployment changed from the traditional "Kubernetes the Hard Way" approach to the **modern kubeadm approach**.

So our documentation must reflect **what we actually deployed**, not the original manual control-plane setup.

Stage 7 — Prepare the Control Plane for kubeadm

Objective

At this stage, we stop generating certificates and start preparing the control-plane nodes so that `kubeadm` can build the Kubernetes control plane using:

- External etcd
- Existing PKI
- Existing HA Load Balancer
- Existing VIP
- Existing kubeadm configuration

What happened before Stage 7?

At this point, we already have:

```
External etcd Cluster
        ✔

Kubernetes CA
        ✔

API Server Certificate
        ✔

Controller Manager Certificate
        ✔

Scheduler Certificate
        ✔

Admin Certificate
        ✔

Service Account Keys
        ✔

Kubeconfigs
        ✔

Encryption Configuration
        ✔
```

Everything required for Kubernetes already exists.

Now we prepare the node for `kubeadm`.

Why use kubeadm now?

Originally (Kubernetes the Hard Way), we would manually create:

- kube-apiserver.service
- kube-controller-manager.service
- kube-scheduler.service
- manifests

That approach works, but it is no longer the recommended production method.

Instead, we let **kubeadm** generate:

- Static Pod manifests
- kubelet configuration
- Bootstrap configuration
- Cluster configuration

This reduces complexity while still allowing us to use our own PKI and external etcd.

Stage 7.1 — Create Required Directories

Run On

cp01, cp02, cp03

Commands

```
mkdir -p /etc/kubernetes/pki
mkdir -p /etc/kubernetes/pki/etcd
mkdir -p /etc/kubernetes/manifests
```

Verify

```
tree /etc/kubernetes
```

Expected:

```
/etc/kubernetes
├── manifests
└── pki
    └── etcd
```

Theory

`kubeadm` expects a standard directory structure.

```
/etc/kubernetes/
```

is Kubernetes' configuration directory.

Inside it:

```
pki/
```

contains certificates.

```
manifests/
```

contains Static Pod manifests.

The kubelet watches this directory continuously.

Whenever a YAML file appears here, kubelet automatically starts that Pod.

This is how:

- kube-apiserver
- kube-controller-manager
- kube-scheduler

will eventually run.

Stage 7.2 — Copy Kubernetes PKI

Run On

cp01

We already generated these certificates in Stage 5.

Copy them into kubeadm's expected location.

```
cp /root/k8s-ha-deployment/04-kubernetes/ca/ca.pem \
/etc/kubernetes/pki/ca.crt

cp /root/k8s-ha-deployment/04-kubernetes/ca/ca-key.pem \
/etc/kubernetes/pki/ca.key

cp /root/k8s-ha-deployment/04-kubernetes/certs/kube-apiserver.pem \
/etc/kubernetes/pki/apiserver.crt

cp /root/k8s-ha-deployment/04-kubernetes/certs/kube-apiserver-key.pem \
/etc/kubernetes/pki/apiserver.key

cp /root/k8s-ha-deployment/04-kubernetes/certs/sa.pem \
/etc/kubernetes/pki/sa.pub

cp /root/k8s-ha-deployment/04-kubernetes/certs/sa-key.pem \
/etc/kubernetes/pki/sa.key
```

Verify

```
ls -lh /etc/kubernetes/pki
```

Expected:

```
ca.crt
ca.key
apiserver.crt
apiserver.key
sa.pub
sa.key
```

Theory

`kubeadm` automatically checks `/etc/kubernetes/pki`.

If certificates already exist there:

- it **uses them**
- it **does not regenerate them**

This is how we keep our custom PKI.

Stage 7.3 — Copy External etcd Certificates

Why?

Our etcd cluster is running on:

- etcd01
- etcd02
- etcd03

The API Server needs three things to connect securely:

1. etcd CA certificate
2. apiserver-etcd-client certificate
3. apiserver-etcd-client private key

These files were generated on **etcd01** during Stage 4, so they must be copied from the etcd cluster to the control-plane nodes.

Run On

First: etcd01

Copy the files to cp01:

```
scp /root/k8s-ha-deployment/03-etcd/ca/ca.pem \
root@10.0.0.11:/tmp/etcd-ca.crt

scp /root/k8s-ha-deployment/03-etcd/certs/apiserver-etcd-client.pem \
root@10.0.0.11:/tmp/apiserver-etcd-client.crt

scp /root/k8s-ha-deployment/03-etcd/certs/apiserver-etcd-client-key.pem \
root@10.0.0.11:/tmp/apiserver-etcd-client.key
```

> **Note:** In our deployment, `/root/k8s-ha-deployment/03-etcd` existed only on the etcd nodes, not on `cp01`. That is why we had to transfer these files using `scp`.

Then: cp01

Move the files into the location expected by kubeadm:

```
mv /tmp/etcd-ca.crt \
/etc/kubernetes/pki/etcd/ca.crt

mv /tmp/apiserver-etcd-client.crt \
/etc/kubernetes/pki/etcd/apiserver-etcd-client.crt

mv /tmp/apiserver-etcd-client.key \
/etc/kubernetes/pki/etcd/apiserver-etcd-client.key
```

Verify

```
ls -lh /etc/kubernetes/pki/etcd
```

Expected:

```
ca.crt
apiserver-etcd-client.crt
apiserver-etcd-client.key
```

Theory

The API Server will use:

```
apiserver-etcd-client.crt
```

to authenticate itself to the external etcd cluster.

Without these files, the API Server cannot read or write cluster state.

Stage 7.4 — Create the kubeadm Configuration File

This is the most important file in the deployment.

Instead of passing many flags to `kubeadm init`, we define everything in a single YAML file.

It includes:

- Kubernetes version
- Cluster name
- Control-plane endpoint (VIP)
- Pod CIDR
- Service CIDR
- External etcd endpoints
- Paths to all certificates
- API Server extra arguments

This file makes the deployment repeatable and easy to version-control.

> We will dedicate **Stage 8** to building and explaining this file in detail.

Stage 7 Summary

At the end of Stage 7:

```
External etcd
        |
        ▼
Certificates copied
        |
        ▼
/etc/kubernetes/pki
        |
        ▼
kubeadm configuration ready
        |
        ▼
Ready for kubeadm init
```

Directory structure on `cp01`

```
/etc/kubernetes/
├── manifests/
└── pki/
    ├── ca.crt
    ├── ca.key
    ├── apiserver.crt
    ├── apiserver.key
    ├── sa.key
    ├── sa.pub
    └── etcd/
        ├── ca.crt
        ├── apiserver-etcd-client.crt
        └── apiserver-etcd-client.key
```

Learning Notes (CKA Perspective)

By the end of Stage 7, you should clearly understand:

- Why `kubeadm` checks `/etc/kubernetes/pki`.
- Why external etcd certificates must come from the etcd cluster.

- Why `apiserver-etcd-client` is different from the API Server TLS certificate.
- Why the API Server is the **client** when communicating with etcd.
- How kubeadm can reuse an existing PKI instead of generating new certificates.

The next stage (Stage 8) focuses entirely on the **`kubeadm-config.yaml`**, which is the blueprint that tells kubeadm exactly how to build your highly available Kubernetes control plane.

This is arguably the **most important stage** in the entire deployment.

Everything we've done so far (HAProxy, Keepalived, External etcd, PKI, kubeconfigs) comes together in a single file.

If you understand this file, you'll understand how `kubeadm` builds a production HA cluster.

Stage 8 — Create the kubeadm Configuration File

Objective

Instead of running:

```
kubeadm init \
--apiserver-advertise-address=... \
--pod-network-cidr=... \
--service-cidr=... \
--control-plane-endpoint=...
```

(which becomes messy and cannot express external etcd or advanced options),

we use a declarative configuration file:

```
kubeadm-config.yaml
```

This file defines the entire cluster.

Think of it as:

> **Terraform for kubeadm.**

Why use a kubeadm config file?

It provides:

- Reproducible deployments
- Version-controlled cluster configuration
- Easier upgrades
- Cleaner HA configuration
- Support for external etcd
- Support for custom certificates
- Support for API Server extra arguments

This is the recommended approach for production.

Run On

cp01 only

Stage 8.1 — Create the Configuration Directory

```
mkdir -p /root/k8s-ha-deployment/05-kubeadm

cd /root/k8s-ha-deployment/05-kubeadm
```

Stage 8.2 — Create `kubeadm-config.yaml`

```
vim kubeadm-config.yaml
```

Paste:

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration

clusterName: pico-prod
kubernetesVersion: v1.31.14

controlPlaneEndpoint: "10.0.0.100:6443"

networking:
  podSubnet: "192.168.0.0/16"
  serviceSubnet: "10.96.0.0/12"
  dnsDomain: "cluster.local"

apiServer:
  certSANs:
    - 10.0.0.100
    - 10.0.0.11
    - 10.0.0.12
    - 10.0.0.13
    - 127.0.0.1

  extraArgs:
    - name: encryption-provider-config
      value: /etc/kubernetes/encryption-config.yaml

etcd:
  external:
    endpoints:
      - https://10.0.0.21:2379
      - https://10.0.0.22:2379
      - https://10.0.0.23:2379

    caFile: /etc/kubernetes/pki/etcd/ca.crt
    certFile: /etc/kubernetes/pki/etcd/apiserver-etcd-client.crt
    keyFile: /etc/kubernetes/pki/etcd/apiserver-etcd-client.key

> **Editorial correction:** the original notes had `extraArgs` written as a flat `key: value` map. That was the `v1beta3` syntax. Under `kubeac

---
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration

cgroupDriver: systemd
```

Understanding Every Section

Cluster Name

```
clusterName: pico-prod
```

This is the logical name of the Kubernetes cluster.

It appears in kubeconfigs and cluster metadata.

Kubernetes Version

```
kubernetesVersion: v1.31.14
```

Locks kubeadm to the desired version.

This prevents accidentally deploying a different release.

Control Plane Endpoint

```
controlPlaneEndpoint:
    10.0.0.100:6443
```

This is **NOT** the API Server itself.

It is the **VIP**.

```
Worker
   |
   ▼
10.0.0.100
   |
HAProxy
   |
cp01 / cp02 / cp03
```

Workers never connect directly to cp01.

They always use the VIP.

This is the core of High Availability.

Pod Network

```
podSubnet:
192.168.0.0/16
```

This range is used by Calico.

Every node receives a subnet from this range.

Example

```
worker01

192.168.5.0/26

worker02

192.168.30.64/26
```

Exactly what we observed after deployment.

Service Network

```
serviceSubnet:
10.96.0.0/12
```

This network is reserved for ClusterIP Services.

Examples

```
kubernetes

10.96.0.1

CoreDNS

10.96.x.x
```

Pods never receive addresses from this range.

DNS Domain

```
cluster.local
```

Creates names like

```
nginx.default.svc.cluster.local
```

API Server Section

```
apiServer:
```

This configures the API Server.

---

certSANs

```
certSANs
```

Adds extra addresses to the API Server certificate.

These must match the certificate we generated manually.

```
VIP

10.0.0.100

cp01

10.0.0.11

cp02

10.0.0.12

cp03

10.0.0.13
```

If these do not match the certificate, TLS verification fails.

---

encryption-provider-config

```
encryption-provider-config:
```

Points to

```
/etc/kubernetes/encryption-config.yaml
```

Without this:

Secrets stored in etcd

↓

Base64 only

With it:

Secrets

↓

AES encryption

↓

etcd

External etcd

```
etcd:
 external:
```

This tells kubeadm

Do NOT create an embedded etcd.

Instead use

```
etcd01

etcd02

etcd03
```

This is exactly our architecture.

---

endpoints

```
https://10.0.0.21:2379

https://10.0.0.22:2379

https://10.0.0.23:2379
```

The API Server connects here.

If one etcd node dies

↓

Remaining two continue serving requests.

Certificates

```
caFile

certFile

keyFile
```

These are the client credentials the API Server uses when talking to etcd.

Remember:

API Server

↓

Client

↓

External etcd

So it must authenticate itself.

Kubelet Configuration

```
cgroupDriver:
systemd
```

Containerd

↓

systemd

↓

kubelet

All three must use the same cgroup driver.

Ubuntu 24.04 defaults to systemd.

Verify

```
cat kubeadm-config.yaml
```

Verify

- VIP is correct
- etcd IPs are correct
- Kubernetes version is correct
- Pod subnet matches Calico (`192.168.0.0/16`)
- Service subnet matches (`10.96.0.0/12`)

Stage 8 Summary

This single file tells kubeadm:

- Which Kubernetes version to install.
- Which VIP clients should use.
- Which Pod CIDR Calico will manage.
- Which Service CIDR Kubernetes will allocate.
- Where the external etcd cluster is located.
- Which certificates to use for etcd authentication.
- Which SANs the API Server certificate must support.
- Where the Secret encryption configuration is stored.
- Which cgroup driver kubelet should use.

This is the **blueprint of your cluster**.

What's Next?

Stage 9 is where everything comes together:

- Validate prerequisites.
- Run `kubeadm init --config=kubeadm-config.yaml`.
- Understand every phase of the initialization process.
- Verify the generated static Pod manifests.
- Configure `kubectl`.
- Join `cp02` and `cp03` as additional control-plane nodes.

- Join the worker nodes.
- Install Calico and verify the entire HA cluster.

This stage transforms your prepared infrastructure into a functioning Kubernetes cluster.

Now we move to the stage where everything comes together.

Until Stage 8 we have:

- ✅ HA Load Balancer (HAProxy + Keepalived)
- ✅ External etcd Cluster
- ✅ Kubernetes PKI
- ✅ kubeconfigs
- ✅ kubeadm configuration
- ✅ First Control Plane initialized

Now Stage 9 is about **building the Highly Available Control Plane**.

Stage 9 — Build the Highly Available Control Plane

Goal

Transform

```
cp01
```

into

```
cp01
cp02
cp03
```

sharing

- External etcd
- VIP
- same CA
- same Service Account keys
- same cluster

Architecture



Why kubeadm join?

When running

```
kubeadm join --control-plane
```

kubeadm automatically

- installs API Server
- installs Controller Manager
- installs Scheduler
- generates kubelet certificates
- creates static manifests
- connects to etcd
- joins the HA cluster

Without kubeadm, every component would need to be created manually.

Stage 9.1 — Upload Cluster Certificates

Run only on

```
cp01
```

Why?

cp02 and cp03 require the same

```
CA
Front Proxy CA
SA keys
```

Instead of copying them manually,

kubeadm encrypts them into a Secret.

Run

```
kubeadm init phase upload-certs --upload-certs
```

Example output

```
[upload-certs] Storing the certificates in Secret "kubeadm-certs"

certificate key:

5c8c31c0d08f7d2b2d...
```

Save this key.

You'll need it when joining cp02 and cp03.

Theory

This certificate key decrypts

```
kubeadm-certs
```

stored inside the cluster.

Without it,

new control-plane nodes cannot obtain

```
CA
SA
Front Proxy
```

Stage 9.2 — Generate Join Command

Run on

```
cp01
```

```
kubeadm token create --print-join-command
```

Example

```
kubeadm join 10.0.0.100:6443 \
--token abcdef.0123456789abcdef \
--discovery-token-ca-cert-hash sha256:xxxxxxxx
```

Save it.

Theory

The join command contains

- API endpoint
- bootstrap token
- cluster CA hash

Workers and control planes both use this.

Stage 9.3 — Join cp02

Run on

```
cp02
```

Example

```
kubeadm join 10.0.0.100:6443 \
--token abcdef.0123456789abcdef \
--discovery-token-ca-cert-hash sha256:xxxxxxxx \
--control-plane \
--certificate-key 5c8c31c0d08f...
```

What happens internally?

kubeadm

↓

Downloads CA

↓

Downloads SA keys

↓

Creates

```
kube-apiserver.yaml

kube-controller-manager.yaml

kube-scheduler.yaml
```

↓

Starts kubelet

↓

kubelet launches

```
API Server
Scheduler
Controller Manager
```

↓

Node joins cluster.

Verify

On cp01

```
kubectl get nodes
```

Expected

```
cp01 Ready
cp02 Ready
```

Stage 9.4 — Join cp03

Run on

```
cp03
```

```
kubeadm join 10.0.0.100:6443 \
--token abcdef.0123456789abcdef \
--discovery-token-ca-cert-hash sha256:xxxxxxxx \
--control-plane \
--certificate-key 5c8c31c0d08f...
```

Wait

```
2-3 minutes
```

Verify

```
kubectl get nodes
```

Expected

```
cp01 Ready
cp02 Ready
cp03 Ready
```

Stage 9.5 — Verify API Servers

Run

```
kubectl get pods -n kube-system -o wide
```

Expected

```
kube-apiserver-cp01

kube-apiserver-cp02

kube-apiserver-cp03
```

All Running.

Verify Scheduler

```
kube-scheduler-cp01

kube-scheduler-cp02

kube-scheduler-cp03
```

Verify Controller Manager

```
kube-controller-manager-cp01

kube-controller-manager-cp02

kube-controller-manager-cp03
```

Stage 9.6 — Verify etcd Connectivity

Run

```
kubectl get componentstatuses
```

(or, on newer Kubernetes versions where this is deprecated, prefer:)

```
kubectl cluster-info
```

Then verify API health:

```
kubectl get --raw='/readyz?verbose'
```

You should see checks such as:

```
etcd ok
poststarthook ok
scheduler ok
controller-manager ok
```

This confirms the API server can communicate with the external etcd cluster and its internal components are healthy.

Stage 9.7 — Test Control Plane Failover

Current

```
VIP

↓

cp01
cp02
cp03
```

Stop API Server on

```
cp01
```

```
mv /etc/kubernetes/manifests/kube-apiserver.yaml \
/root/
```

Wait

```
30 seconds
```

Check

```
kubectl get nodes
```

It should still work because HAProxy routes traffic to cp02 or cp03.

Restore

```
mv /root/kube-apiserver.yaml \
/etc/kubernetes/manifests/
```

Theory Recap

By the end of Stage 9, you have:

- **Three control-plane nodes** (`cp01` , `cp02` , `cp03`) serving the Kubernetes API.
- **HAProxy + Keepalived** exposing a single VIP (`10.0.0.100:6443`) so clients and workers always use one endpoint.
- **External etcd cluster** storing all Kubernetes state independently of the control-plane nodes.
- **Shared PKI and Service Account keys** distributed securely using `kubeadm upload-certs` .
- **Automatic failover**: if one control-plane node fails, HAProxy forwards traffic to the remaining healthy API servers without requiring any changes on worker nodes.

The next stage is **Stage 10 — Joining Worker Nodes**, where you'll securely join `worker01` through `worker05` , verify kubelet registration, deploy Calico networking, and validate end-to-end pod scheduling and inter-node communication.

Stage 10 — Join Worker Nodes & Build the Kubernetes Data Plane

Objective

In Stage 9, we built the **Highly Available Control Plane**.

Now we build the **Data Plane** by joining the worker nodes.

By the end of this stage:

- ✅ worker01–worker05 join the cluster
- ✅ kubelet registers each worker with the API Server
- ✅ kube-proxy runs on every node
- ✅ Calico runs on every node
- ✅ Pod networking works across all nodes
- ✅ Workloads can be scheduled on worker nodes

Architecture Before Stage 10

```
               VIP
         10.0.0.100:6443
               |
     HAProxy + Keepalived
               |
     ------------------------
     |         |          |
   cp01      cp02       cp03
     \        |         /
      \       |        /
       External etcd Cluster
```

Workers exist physically but are **not yet part of the Kubernetes cluster**.

What happens when a Worker joins?

When you execute

```
kubeadm join ...
```

the following happens:

```
Worker
   |
   ▼
Connects to VIP (10.0.0.100)
   |
   ▼
HAProxy forwards request
   |
   ▼
API Server authenticates bootstrap token
   |
   ▼
CSR (Certificate Signing Request) is generated
   |
   ▼
API Server signs kubelet certificate
   |
   ▼
Worker becomes Ready
```

Stage 10.1 — Generate Worker Join Command

Run ONLY on

```
cp01
```

Generate a new join command:

```
kubeadm token create --print-join-command
```

Example:

```
kubeadm join 10.0.0.100:6443 \
--token abcdef.0123456789abcdef \
--discovery-token-ca-cert-hash sha256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Theory

The command contains:

- Cluster endpoint (VIP)
- Bootstrap token
- CA certificate hash

This securely verifies that the worker is joining the correct cluster.

Stage 10.2 — Join Worker Nodes

Run on

```
worker01
```

```
kubeadm join 10.0.0.100:6443 \
--token abcdef.0123456789abcdef \
--discovery-token-ca-cert-hash sha256:xxxxxxxx
```

Repeat the same command on:

```
worker02

worker03

worker04

worker05
```

What happens internally?

```
Worker
│
├── kubeadm downloads cluster information
│
├── kubelet starts
│
├── kubelet generates private key
│
├── kubelet creates CSR
│
├── API Server signs certificate
│
├── kubelet.conf created
│
└── Node registers
```

Stage 10.3 — Verify Worker Registration

Run on

```
cp01
```

```
kubectl get nodes
```

Expected output:

```
NAME        STATUS   ROLES          VERSION
cp01        Ready    control-plane
cp02        Ready    control-plane
cp03        Ready    control-plane
worker01    Ready    <none>
worker02    Ready    <none>
worker03    Ready    <none>
worker04    Ready    <none>
worker05    Ready    <none>
```

More detailed output:

```
kubectl get nodes -o wide
```

Example:

```
NAME       STATUS   ROLES           INTERNAL-IP
cp01       Ready    control-plane   10.0.0.11
cp02       Ready    control-plane   10.0.0.12
cp03       Ready    control-plane   10.0.0.13
worker01   Ready    <none>          10.0.0.31
worker02   Ready    <none>          10.0.0.32
worker03   Ready    <none>          10.0.0.33
worker04   Ready    <none>          10.0.0.34
worker05   Ready    <none>          10.0.0.35
```

Stage 10.4 — Verify Certificate Signing Requests (CSR)

Run on

```
cp01
```

```
kubectl get csr
```

Expected:

```
Approved,Issued
```

Example:

```
NAME        CONDITION
csr-abc     Approved,Issued
csr-def     Approved,Issued
```

Theory

Every kubelet receives its own unique client certificate.

The certificate is later used for:

- Authentication
- Secure communication
- Node identity

Workers never share certificates.

Stage 10.5 — Verify kube-system Pods

Run:

```
kubectl get pods -n kube-system -o wide
```

You should see:

```
kube-proxy
```

running on

```
cp01
cp02
cp03
worker01
worker02
worker03
worker04
worker05
```

If Calico is already installed, you should also see:

```
calico-node
```

running on every node.

Theory

kube-proxy

Responsible for:

- Service networking
- ClusterIP
- NodePort
- iptables/IPVS rules

Runs on every node.

---

calico-node

Responsible for:

- Pod networking
- Pod routing
- Network policy
- IP allocation (IPAM)

Runs as a DaemonSet on every node.

Stage 10.6 — Verify PodCIDR Assignment

Run:

```
kubectl get nodes \
-o custom-columns=NAME:.metadata.name,PODCIDR:.spec.podCIDR
```

Example:

```
cp01        192.168.214.128/26
cp02        192.168.63.64/26
cp03        192.168.31.128/26
worker01    192.168.5.0/26
worker02    192.168.30.64/26
worker03    192.168.19.64/26
worker04    192.168.196.64/26
worker05    192.168.99.128/26
```

Theory

The Controller Manager allocates a unique PodCIDR to each node.

Example:

```
worker01

PodCIDR

192.168.5.0/26
```

Pods on worker01 receive IPs from this range only.

Stage 10.7 — Verify Calico Routing

Run on any worker:

```
ip route
```

Example:

```
192.168.19.64/26 via 10.0.0.33 dev tunl0

192.168.30.64/26 via 10.0.0.32 dev tunl0

192.168.214.128/26 via 10.0.0.11 dev tunl0
```

Theory

Each route represents another node’s PodCIDR.

```
Pod on worker01

↓

tunl0

↓

worker03

↓

Destination Pod
```

Calico creates these routes automatically.

Stage 10.8 — Verify kubelet Health

Run on every worker:

```
systemctl status kubelet --no-pager
```

Expected:

```
Active: active (running)
```

Verify runtime:

```
crictl ps -a
```

You should see:

```
kube-proxy

calico-node
```

running.

Stage 10.9 — Verify Cluster Health

Run on cp01:

```
kubectl cluster-info
```

Expected:

```
Kubernetes control plane is running at

https://10.0.0.100:6443
```

Run:

```
kubectl get componentstatuses
```

> Note: `kubectl get componentstatuses` is deprecated in recent Kubernetes releases. A more current alternative is:
>
> ```
> kubectl get --raw='/readyz?verbose'
> ```
>
> or simply:
>
> ```
> kubectl cluster-info
> ```

Run:

```
kubectl get pods -A
```

Everything should be in the `Running` state (except any completed Jobs, which may show `Completed`).

Stage 10 Summary

At the end of Stage 10, your cluster consists of:



You now have a fully functional **production-style HA Kubernetes cluster** with:

- 3 Control Plane nodes
- 3 External etcd nodes
- 2 Load Balancers with HAProxy + Keepalived
- 5 Worker nodes
- kube-proxy on every node
- Calico networking across the cluster
- Secure TLS communication
- High availability for both the API server and etcd

The next stage, **Stage 11**, focuses on **deploying applications, validating scheduling, testing networking, and performing failover and recovery tests** to confirm that the cluster behaves correctly under real-world conditions.